RELAYSPEC: RETARGETING FROZEN SPECULATIVE DRAFTERS WITH SMALL-DATA LINEAR INTERFACES

Anonymous authors
Paper under double-blind review

ABSTRACT

Feature-conditioned speculative drafters depend on the hidden states of the model used to train them. Reusing such a drafter with a larger target can therefore require running its source model alongside that target. RelaySpec replaces this dependency with a learned linear interface while keeping the target and drafter frozen. Across DFlash and EAGLE-3 drafters retargeted from Qwen3-4B to 8B and 14B, the original 4,096-record fits achieve 2.35–5.11 times autoregressive throughput on MATH-500. Separate EAGLE-3 measurements reduce peak allocated memory by 7.63–7.80 GiB relative to source reuse. Controlled scaling studies show that the interface can be learned from substantially fewer records: 512-example maps retain 97.8% (DFlash) and 98.3% (EAGLE-3) of their respective 2,048-example maps' throughput on 128 development questions with 8B targets. Increasing mapper capacity improves feature fitting initially, but wider MLPs and longer training do not consistently improve decoding. Under matched warm-training budgets, feature regression outperforms the tested connector-CE and LoRA alternatives. These results identify a low-cost reuse regime and its limits, including task-dependent slowdowns relative to source reuse, optimization sensitivity and finite-precision output differences.

1 INTRODUCTION

A developer may have a DFlash drafter trained for Qwen3-4B but want to serve Qwen3-8B or Qwen3-14B. The new target can check its proposals, but supplies different hidden states. This creates a deployment choice: train a new target-specific drafter, or reuse the existing drafter while paying for its source model to provide compatible features. RelaySpec addresses deployments that seek to reuse this training investment with limited additional fitting work. It learns a replacement for the source conditioning path, allowing the new target to supply the drafter's input. The practical goal is faster generation than plain AR while retaining much of a native target-specific drafter's throughput.

Speculative decoding accelerates autoregressive (AR) generation by checking several proposed tokens in one target pass (Leviathan et al., 2023). DFlash and EAGLE-3 improve proposals using features computed inside a language model (Chen et al., 2026; Li et al., 2025). We call the model that provided these features during drafter training the *source*. The *target* is the model whose token choices govern deployment. Running the source alongside a new target supplies compatible features, but also retains its transformer computation and memory.

RelaySpec runs the source and new target on shared text during fitting. It learns a linear map from the target's hidden states to the context that the source supplies to the drafter. At inference, the target already computes these states when checking proposals. Applying the map supplies the next drafting input without another source-transformer pass. The existing target verifier still decides which tokens to commit.

Only the map is trained. The target, drafter, inherited embedding and vocabulary projection stay fixed. We apply this procedure to DFlash and EAGLE-3, matching each released decoder's input normalization. We measure speed relative to AR (Figure 1), native-drafter throughput retained, and additional fitting data and work. Source reuse isolates source-removal savings. Our controlled studies then separate training-data count, optimization work and mapper capacity. They show that 512-record fits retain most of the measured 2,048-record throughput in both 8B drafter families, while better training-feature fit need not improve decoding. Matched warm-training budgets compare feature regression with connector CE and drafter LoRA. Together these experiments identify when a small interface is sufficient and where its benefits end.

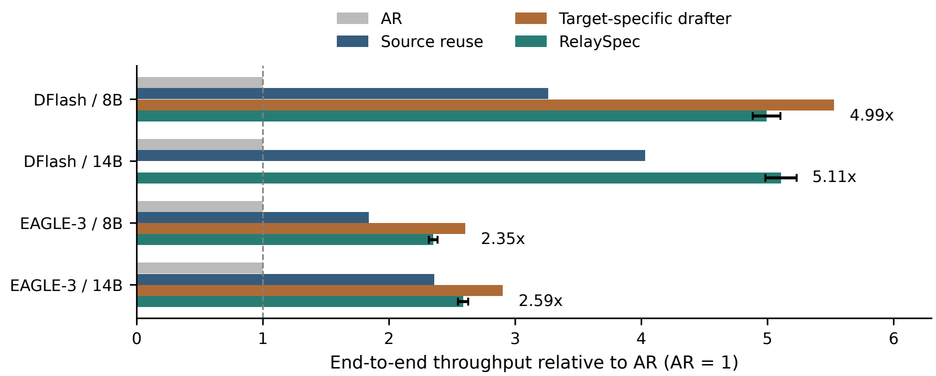

Figure 1: MATH-500 throughput relative to plain AR, measured within each paired run. RelaySpec reuses the 4B drafter. The native control uses a drafter released for the evaluated target. Whiskers show RelaySpec's 95% paired intervals. Native controls show the three evaluated releases. Each runtime uses one request per GPU.

2 Related work and positioning

Reuse and adaptation are different deployment choices. DFlash and EAGLE-3 train drafters using their target's internal features (Chen et al., 2026; Li et al., 2025). RelaySpec inherits such a trained component and changes only its input interface. PARD builds a parallel drafter independently of a particular target (An et al., 2026). SD^2 learns target-feature steering inside drafter MLPs, including frozen-drafter variants (Berdoz et al., 2026). Its token-distribution objective differs from regressing the source context already consumed by an inherited feature-conditioned drafter. OmniDraft adapts online through hybrid distillation and vocabulary handling (Ramakrishnan et al., 2025). Our setting is offline retargeting within a shared vocabulary. Heterogeneous verification addresses other vocabulary settings (Timor et al., 2025a).

Interfaces between frozen models. Adapters, low-rank updates and model stitching already connect or adapt frozen networks (Houlsby et al., 2019; Hu et al., 2022; Bansal et al., 2021). Our contribution is identifying and replacing the source-conditioning interface in existing speculative decoders, then measuring the data, capacity and runtime tradeoffs of doing so. Feature agreement does not establish equivalent representations (Smith et al., 2025). We therefore evaluate actual decoding, compare equal-capacity linear and nonlinear maps, and test cheap drafter adaptation under measured training budgets. Appendix H discusses broader methods. Public SD^2 and PARD evaluations are reported separately in the appendix because their runtime and inherited models differ.

3 Preliminaries

3.1 Drafting and target verification

An autoregressive model generates one token at a time. Blockwise and speculative methods propose several tokens, then check them in a target pass (Stern et al., 2018; Xia et al., 2023; Leviathan et al., 2023). In greedy decoding, the verifier commits the longest matching prefix and a target-selected next token. RelaySpec preserves this decoder and changes the features supplied to its drafter.

3.2 THROUGHPUT AND ACCEPTED PROGRESS

Our primary metric is end-to-end output throughput: total generated tokens divided by total request time, including prompt processing. Let T_A, T_R and T_N denote this throughput for plain AR, RelaySpec and the native target-specific drafter. We report

$$\text{AR speedup} = T_R/T_A, \qquad \text{native throughput retained} = 100\, T_R/T_N\ \%. \tag{1}$$

A retention value of 90% means the reused drafter produces 90% as many tokens per second as the target's own drafter. It is not the fraction of speedup above AR. We also report the ratio of summed AR request time to summed relay request time. This differs from a throughput ratio when output lengths differ.

Acceptance explains how speed is obtained. We report the pooled number of tokens advanced per proposal-verification cycle, including the implementation's target-supplied progress. This is distinct from the fraction of proposed draft tokens accepted. Appendix E gives the position-wise curves and precise counting convention.

Let c_R be the average relay cycle time and a_R its average progress. Its decode time per output token is approximately c_R/a_R. If c_A is AR time per token, then the decode-only approximation is

$$\text{AR-relative decode speed} \ \approx\ \frac{a_R c_A}{c_R}. \tag{2}$$

Removing source conditioning reduces cycle cost. Imperfect feature matching may reduce progress. The map is useful when the saved computation outweighs the extra cycles. We measure end-to-end speed directly and use this approximation to interpret the component measurements.

4 METHOD

4.1 FIT THE INPUT CONSUMED BY THE FROZEN DRAFTER

We run the source and new target on the same text and collect hidden states after five selected transformer blocks. Concatenating these states gives the target vector h_t and source vector s_t at position t. The drafter's frozen projection F converts source features to its input width. We learn a bias-free matrix W that supplies this input from h_t. Figure 2 separates fitting from generation.

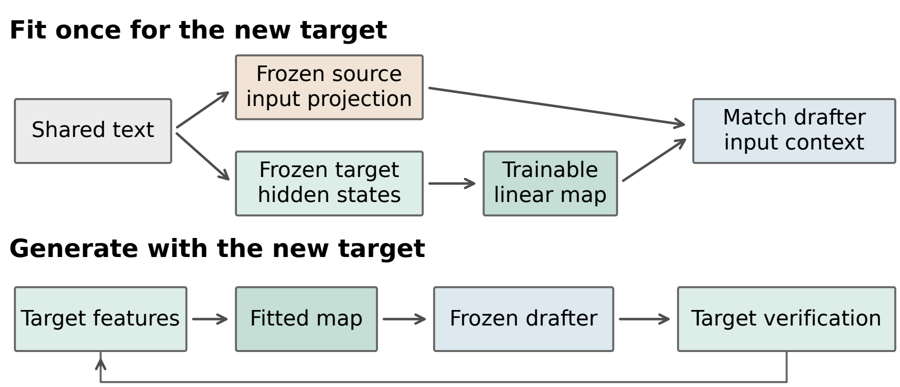

Figure 2: Fit once, then reuse the frozen drafter. The source and target process identical text during fitting. Only the map is updated. During generation, target features from committed positions feed the next draft.

The correct input depends on the released drafter. DFlash consumes a normalized projection. Let N_{in} denote fixed root-mean-square normalization of joined target features (Zhang & Sennrich, 2019), which rescales a vector by the square root of its mean squared coordinate. Let N_{out} be the released drafter's frozen output normalization. EAGLE-3 consumes the raw projection. The teacher context c_t

and mapped context $\widehat{c}_t$ are

$$\begin{aligned} \text{DFlash:} \quad & c_t = N_{\text{out}}(Fs_t), \quad & \widehat{c}_t = N_{\text{out}}(WN_{\text{in}}(h_t)), \\ \text{EAGLE-3:} \quad & c_t = Fs_t, & \widehat{c}_t = Wh_t. \end{aligned} \tag{3}$$

Preserving feature magnitude matters for the raw EAGLE-3 input. DFlash's output normalization is applied to both teacher and prediction. The normalization ablation tests these choices directly.

We fit W by minimizing the squared context error relative to the teacher's squared magnitude. For a record with n token positions,

$$\mathcal{L}(W) = \frac{1}{n} \sum_{t=1}^{n} \frac{\|\widehat{c}_t - c_t\|_2^2}{\max(\|c_t\|_2^2, \epsilon)}. \tag{4}$$

Here $\|v\|_2^2$ sums the squared coordinates of v, and ϵ is the smallest positive normal FP32 value. Dividing by teacher magnitude prevents larger context vectors from dominating the objective. We average positions within each record and record losses across workers. DFlash's normalization makes this objective different from raw linear least squares. Both selected maps are fitted with gradient updates.

The original deployment evaluation uses 4,096 distinct math problem-and-solution records rendered as conversations, with the resulting text truncated to 192 tokens. The controlled scaling studies vary this fitting budget as specified below. The supervision is a source context vector. Solution text remains part of the input. The fitted map has 52.43M weights for an 8B target and 65.54M for a 14B target. Appendix A specifies the layers and dimensions.

4.2 Draft with the new target's features

The target first processes the prompt and retains its attention cache and selected hidden states. The map supplies drafter context. The frozen drafter proposes tokens, and the target checks them in parallel with a causal mask. The decoder commits the longest matching prefix and the target-selected next token, then crops the cache to the committed prefix. Mapped target states supply the next cycle, as shown in Figure 6.

The standard greedy equivalence argument requires block verification to return the same token choices as single-token evaluation of each identical prefix (Leviathan et al., 2023). Finite-precision implementations can change those choices. We therefore measure task accuracy and token agreement separately from throughput. The map itself never supplies the scores used for target verification.

5 Experiments

5.1 Experimental setup

Models and runtime. We retarget the released Qwen3-4B DFlash and DeepSpec EAGLE-3 drafters to Qwen3-8B and Qwen3-14B (Yang et al., 2025; DeepSeek-AI, 2026). DFlash uses block length 16. EAGLE-3 uses the supported chain procedure with seven draft positions. Native controls use released target-specific drafters within the same implementation and proposal setting. The main AR comparisons use four NVIDIA L40S GPUs, with one sequential request per worker. Historical block-size and isolated-memory studies use RTX 6000 Ada GPUs. We compare methods within runs on the same hardware. Serving systems such as vLLM and SGLang also optimize cache use and request execution (Kwon et al., 2023; Zheng et al., 2024). Our timing isolates the stated per-request decoder rather than a concurrent serving workload.

Table 1: Evaluation protocol. Main fitting choices remain fixed across workloads. Full identifiers and runtime versions are in Appendix A.

Component	Setting
Primary baseline	Plain greedy AR with the same target, prompt and runtime
Other controls	Native target-specific drafter and optimized source reuse
Generation	BF16, SDPA attention, thinking disabled, maximum 2,048 new tokens
Relay fitting	4,096 records, 192-token cap, 1,024 AdamW updates (Loshchilov & Hutter, 2019), four records/update
Optimizer	Learning rate 6×10^{-4}, zero weight decay, gradient clipping 1.0
Timing	Synchronized request timer, warmup excluded, rotated method order
Uncertainty	10,000 paired resamples, whole conversations for two-turn chat

Workloads and data use. We evaluate math, code and conversation workloads because speculative speed varies across tasks (Xia et al., 2024; Abramovich et al., 2026). MATH-500 supplies 500 questions (Lightman et al., 2024). The breadth suite contains 128 GSM8K questions, 164 HumanEval tasks, 378 MBPP tasks and 80 two-turn MT-Bench conversations (Hendrycks et al., 2021; Cobbe et al., 2021; Chen et al., 2021; Austin et al., 2021; Zheng et al., 2023). The 32-question development set informed interface and loss choices. Historical evaluations retain this exposure history. Fitting and evaluation problem texts have zero normalized exact matches. Appendix F quantifies related templates using token overlap. All completed workload cells in each reported suite are included.

Controlled scaling protocol. The original 4,096-record checkpoints remain fixed for the broad workload results. A fixed-1,024-update sweep varies distinct records from 64 to 4,096. A separate NuminaMath study uses nested 512/2,048-record sets, 1,024 feature-validation records and 8,192 batch-four updates per fit. Frozen feature caches are shared across matched linear and MLP fits. The longer-output capacity comparisons use 128 exposed MATH questions and a 2,048-token cap. The 14B capacity and adaptation screens use sixteen exposed questions and a 256-token cap. These studies have separate checkpoints and are not pooled with the broad evaluation.

Quality and pairing. We score the same math outputs used for the AR timing comparison with the pinned Qwen math scorer. Historical code comparisons use EvalPlus base and expanded tests (Liu et al., 2023). MT-Bench measures two-turn generation time, with each method conditioning its second turn on its own first answer. Its bootstrap unit is the conversation. Other confidence intervals resample matched requests and condition on the fitted checkpoint. Model loading and external task scoring sit outside the request timer.

5.2 Acceleration over plain autoregressive decoding

Table 2 reports absolute end-to-end throughput and AR-relative speed. DFlash RelaySpec reaches 186.49 tokens/s at 8B and 116.88 at 14B, compared with AR at 37.37 and 22.88. EAGLE-3 RelaySpec reaches 84.92 and 59.18 tokens/s, compared with 36.15 and 22.88 for AR. The resulting throughput ratios are 4.99, 5.11, 2.35 and 2.59. Each paired interval is above one.

Table 2: MATH-500 end-to-end tokens/s, including prompt processing. Progress R is mean recorded relay progress per cycle. Ratios and intervals use the AR arm of that same run. Native-drafter comparisons are in Table 3.

Drafter	Target	AR tok/s	Source tok/s	Relay tok/s	Relay / AR [95% CI]	Progress R
DFlash	8B	37.37	121.81	186.49	4.99 [4.88, 5.10]	6.98
DFlash	14B	22.88	92.15	116.88	5.11 [4.98, 5.23]	6.49
EAGLE-3	8B	36.15	66.44	84.92	2.35 [2.32, 2.38]	4.33
EAGLE-3	14B	22.88	53.96	59.18	2.59 [2.55, 2.63]	4.01

DFlash's parallel proposal step and longer accepted progress yield higher throughput than the sequential EAGLE-3 path in these implementations. This is a comparison of the stated released models and decoding settings. The two integrations use different pinned Transformers versions, so their cross-family difference is not an isolated architectural effect.

5.3 Retaining target-specific drafter performance

For the motivating 4B-to-8B case, RelaySpec reaches 186.49 tokens/s against native DFlash-8B's 206.58, retaining 90.3% of its throughput. EAGLE-3 retains 90.2% at 8B and 89.1% at 14B (Table 3). These are within-runtime comparisons to released target-specific drafters. They measure the performance of reuse, not a matched experiment on the cost of originally training those releases. We evaluate marginal adaptation cost directly below.

5.4 Acceptance and the source-removal mechanism

Table 3 pairs retained throughput with accepted progress. RelaySpec advances 6.98 tokens per DFlash-8B cycle versus 7.89 for its native drafter. EAGLE-3 advances 4.33 versus 5.40 at 8B and 4.01 versus 5.28 at 14B. Acceptance and speed retention therefore answer different questions: per-cycle work also changes with drafter size and implementation.

Table 3: Native throughput retained and mean recorded progress per cycle. N and R denote native drafting and RelaySpec. Progress includes the implementation's target-supplied token.

Drafter	Target	Native tok/s	Relay tok/s	Retained [95% CI]	Progress N	Progress R
DFlash	8B	206.58	186.49	90.3% [89.8, 90.8]	7.89	6.98
EAGLE-3	8B	94.12	84.92	90.2% [89.8, 90.7]	5.40	4.33
EAGLE-3	14B	66.39	59.18	89.1% [88.6, 89.7]	5.28	4.01

Against source reuse, the relay's throughput ratios are 1.531 and 1.268 for DFlash, and 1.278 and 1.097 for EAGLE-3, at 8B and 14B respectively. This secondary comparison holds the inherited drafter fixed and isolates source replacement. The source's computation disappears from each cycle, while the target's retained states already contain useful context. The matched EAGLE-3 normalization result in Section 5.8 supports the importance of supplying that context at its expected scale.

5.5 Math accuracy and output agreement

Table 4 scores the exact outputs timed in Table 2. Observed AR-to-relay accuracy differences range from -0.2 to $+0.8$ percentage points, and each paired interval includes zero. These measured outcomes support the accuracy comparison without treating token equality as a substitute for task quality.

Table 4: MATH-500 accuracy in percent. Difference is RelaySpec minus AR in percentage points. Token matches count identical full output sequences.

Drafter	Target	AR score	Relay score	Difference [95% CI]	Token matches
DFlash	8B	75.8	76.6	+0.8 [-1.4, +3.0]	117/500
DFlash	14B	79.2	79.2	+0.0 [-2.2, +2.0]	113/500
EAGLE-3	8B	75.8	75.6	-0.2 [-2.4, +2.0]	135/500
EAGLE-3	14B	79.2	79.4	+0.2 [-1.8, +2.2]	106/500

In BF16, AR and block verification can select different tokens on an identical prefix. Traces of eight selected DFlash early-divergence prompts show changes in the leading target scores shared by native drafting, source reuse and RelaySpec. This diagnostic is separate from the timed runs. We report measured quality and agreement rather than claiming bitwise identity from the verifier's mathematical contract.

5.6 Speed across workloads

Figure 3 and Table 6 report the completed DFlash AR-paired breadth suite. RelaySpec is 1.85–3.69 times faster than AR across these eight cells. Code gains at 14B remain substantial relative to AR even where source reuse is faster than the relay. The useful comparison depends on the deployment choice: AR measures the acceleration obtained, while source reuse measures the benefit of removing its source.

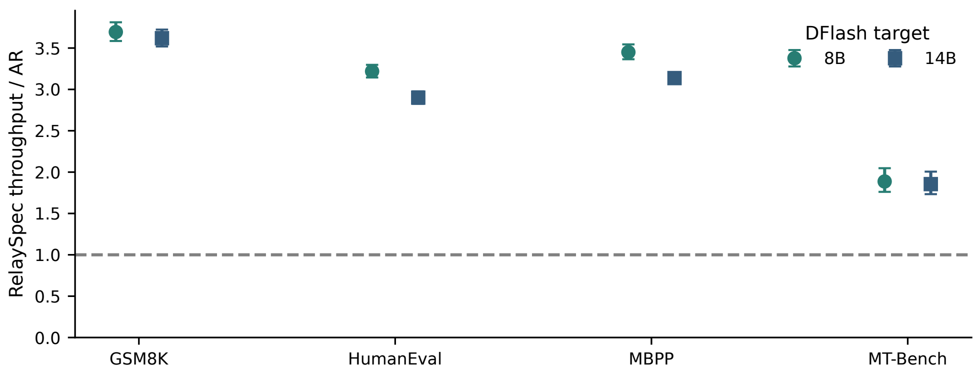

Figure 3: DFlash RelaySpec throughput relative to AR across workloads. Whiskers are paired 95% intervals. Chat intervals resample whole two-turn conversations. Full absolute values are in Table 6.

5.7 Fitting work and deployment memory

The selected maps have 52.43–65.54M trainable weights. Recorded fitting loops take 85.6–118.5 seconds on four RTX 6000 Ada GPUs with models already loaded. The interval covers feature computation and optimization. Loading, checkpoint writing and deployment initialization are separate setup costs. Appendix G gives each measured fitting loop.

Separate EAGLE-3 processes measure memory after removing source layers. Peak allocated memory falls from 25.56 to 17.76 GiB at 8B and from 37.56 to 29.94 GiB at 14B. These are savings of 7.80 and 7.63 GiB. Source token embeddings and vocabulary projections required by a drafter remain part of deployment. The claim concerns the removed source transformer, not every tensor inherited from the source.

5.8 Ablation studies

5.8.1 How much data and mapper capacity are needed?

We separate distinct training examples from optimization work. With 1,024 updates and four examples per update held fixed, DFlash-8B throughput increases from 134.4 tokens/s at 64 examples to 181.9 at 512, then ranges from 183.8 to 184.6 at 1,024–4,096 examples. Thus 512 is the smallest tested count within 5% of the observed best point, not a confirmed data threshold beyond this setting. The right panel of Figure 4 separately follows one continuous fit.

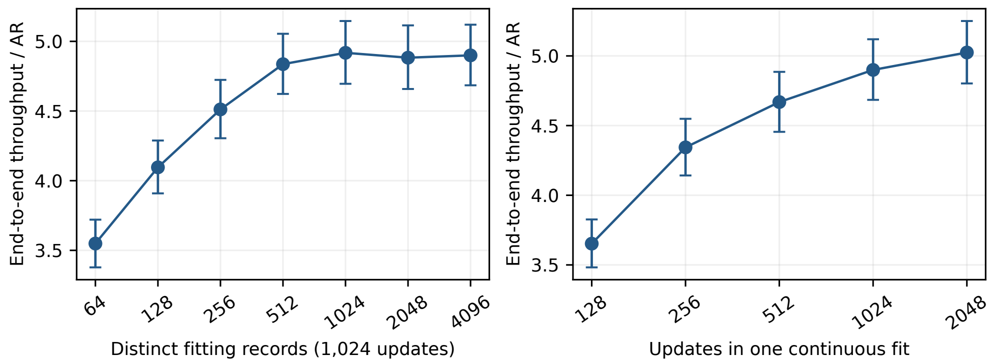

Figure 4: DFlash-8B on 128 development requests. Left: distinct-data scaling at fixed optimization work. Right: checkpoints from one continuous fit. Paired 95% request intervals exclude fitting-seed variation and selection uncertainty.

A separate NuminaMath study crosses 512/2,048 examples with dense, factored linear and GELU MLP maps over seven widths, using 8,192 updates per cell. This holds parameter count equal between each factored map and MLP. Validation error decreases with capacity and then flattens (Figure 5). The measured 128-question, 2,048-token decoding comparison gives dense $N = 512$ 193.68 tokens/s versus 198.09 for dense $N = 2048$, retaining 97.78% [97.09, 98.47]%. Linear-1024 at $N = 512$ retains 95.00% [94.14, 95.85]% with 23.59M parameters instead of 52.43M. The latter interval crosses the 95% boundary. These are development comparisons with fixed fitting seeds.

The completed EAGLE-3 comparison independently measures the same data-count contrast: dense $N = 512$ reaches 93.47 tokens/s versus 95.10 for dense $N = 2048$, retaining 98.28% [97.59, 99.01]%. Linear-2048 at $N = 512$ retains 98.69% [98.08, 99.31]%, while MLP-2048 at $N = 2048$ retains 93.85% [93.27, 94.46]%. All eight maps score 105/128, versus AR's 104/128. Identical paired correctness among these maps still permits a conservative 95% difference interval of ± 3.37 percentage points. Table 21 retains every evaluated method.

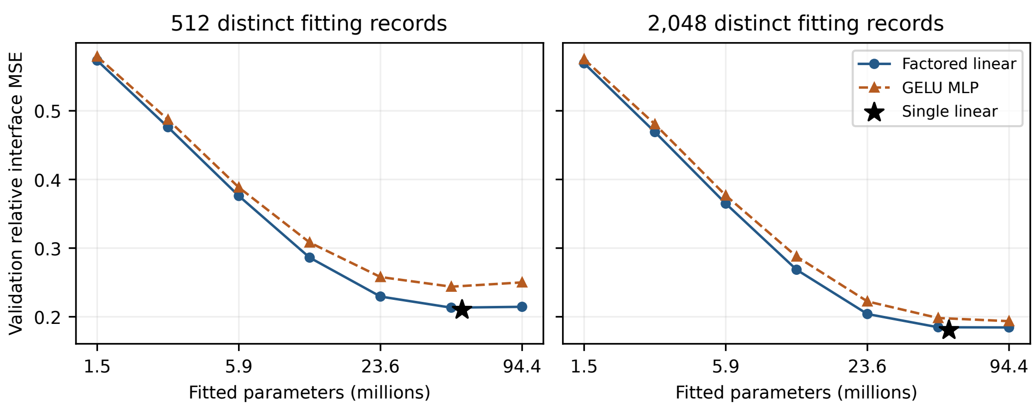

Figure 5: Complete primary capacity study at 8,192 updates per cell. Markers compare validation interface error, with one fitting seed. The curves do not establish decoding speed or answer quality.

Better feature fitting need not mean faster decoding. The 94.37M-parameter MLP-4096 at $N = 2048$ reaches 190.15 tokens/s, below dense. Extending the width-512 MLP from 8,192 to 32,768 updates on the same 2,048 records raises throughput only from 154.39 to 157.68. All tested mappers score 105/128 at this cap, versus AR's 104/128, which does not establish a one-percentage-point noninferiority margin. The regularization study finds no matched endpoint benefit from its L2 settings and only small weight-decay gains in feature validation. Complete positive and negative results appear in the appendix.

Replication and workload dependence. The complete 14B grids repeat the small-data/capacity comparison in both drafter families. DFlash's wide MLP fits its training features better than dense but has worse validation error and lower capped decoding throughput. EAGLE-3 shows late degradation in both dense 512-example seeds and a transient wide-linear loss spike. These results delimit the shared optimization recipe rather than prove a disadvantage of a function class (Appendix C.6). On the 8B capped long-output development set, dense $N = 512$ retains 97.4–97.9% of dense $N = 2048$ throughput across MATH difficulty groups. Very long inputs have too few examples for a reliable generalization claim.

5.8.2 Is feature fitting a useful adaptation budget?

We compare DFlash feature regression with connector cross-entropy and rank-32 drafter LoRA on the same 512 records. An equal three-rate screen sets CE and LoRA rates. Budgets of 22.85, 91.41 and 365.65 warm-training seconds count the initial mapper required by CE and LoRA. At the middle budget, feature regression reaches 144.80 tokens/s, connector CE 109.60, and two LoRA initializations 115.02 and 115.49. Across all three budgets, CE retains 73.2–76.6% and LoRA 79.4–82.7% of feature throughput. This supports the tested reuse procedure under limited update time. It is not a general ranking of LoRA or an end-to-end training-cost comparison: preparation and export are reported separately, and decoding uses sixteen development questions capped at 256 tokens. The appendix provides all budgets, fitting histories and uncertainty intervals.

Public-runtime baselines. On 128 exposed questions capped at 2,048 tokens, PARD reaches 3.34 times its own AR throughput and the tested frozen-drafter SD^2 configuration reaches 1.60 times. Their observed correct counts are 105 versus 103 and 103 versus 103, respectively. Different attention, precision and library settings prevent an algorithm-only ranking against RelaySpec. Appendix D reports full paired results, conservative quality intervals and adaptation settings.

5.8.3 Draft length and interface choices

The block-size experiment directly compares AR, native DFlash and the relay at lengths 8, 16 and 32 on 64 fixed requests. Block 16 gives the highest relay throughput, 210.15 tokens/s (Table 10). Block 8 retains more source progress proportionally but advances fewer tokens per cycle. Block 32 adds verification width while reducing accepted progress for the released block-16 drafter. This supports keeping its trained block length instead of maximizing the relative gain over source reuse.

The EAGLE-3 normalization comparison holds parameter count, data and 128-update budget fixed. Raw target features reduce relative context error from 0.413 to 0.266 and increase the source-relative throughput ratio from 1.035 to 1.237. The raw interface consumes feature magnitude, which input normalization removes. This gives a concrete reason for preserving scale in the selected map.

Appendix C reports the DFlash objective, ridge and normalization comparisons with their distinct fitting recipes.

6 Conclusion

RelaySpec replaces source-model conditioning with a learned input map, reusing frozen speculative drafters across the tested target sizes. The controlled studies identify a small-data regime and diminishing returns from mapper capacity and additional updates. Feature fitting is a useful surrogate, but deployment decisions require measured acceptance, speed and quality. Results concern shared-vocabulary Qwen3 models, greedy decoding and one request per worker. Existing development exposure and limited fitting seeds prevent general minimum-data or noninferiority claims. Code workloads can favor source reuse, and EAGLE-3 optimization is sensitive to the training recipe. Zero exact training overlap does not rule out shared math templates. The controlled 14B and adaptation comparisons remain short-output screens. Neither untouched confirmation nor tight quality noninferiority is established.

AI USE STATEMENT

Generative AI assisted with implementation, code review, literature search, analysis and manuscript editing. Numerical tables and plots are generated from recorded artifacts with checked method, request and scorer identity. Scientific interpretation and submission decisions remain the authors' responsibility.

REPRODUCIBILITY STATEMENT

The appendix specifies model identifiers, feature layers, fitting settings, measurement boundaries, complete workload results and data checks. The accompanying implementation includes configurations, request records, scorer provenance and scripts that regenerate the reported assets.

REFERENCES

Talor Abramovich, Maor Ashkenazi, Izzy Putterman, Benjamin Chislett, Tiyasa Mitra, Bita Darvish Rouhani, Ran Zilberstein, and Yonatan Geifman. SPEED-Bench: A unified and diverse benchmark for speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL `https://arxiv.org/abs/2604.09557`. Accepted paper.

Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost PARallel draft model adaptation. In *International Conference on Learning Representations*, 2026. URL `https://openreview.net/forum?id=XbOyv7iVGL`.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL `https://arxiv.org/abs/2108.07732`.

Yamini Bansal, Preetum Nakkiran, and Boaz Barak. Revisiting model stitching to compare neural representations. In *Advances in Neural Information Processing Systems 34*, 2021. URL `https://papers.nips.cc/paper/2021/hash/01ded4259d101feb739b06c399e9cd9c-Abstract.html`.

Frédéric Berdoz, Peer Rheinboldt, and Roger Wattenhofer. Steering pretrained drafters during speculative decoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 30067–30075, 2026. doi: 10.1609/aaai.v40i36.40255. URL `https://ojs.aaai.org/index.php/AAAI/article/view/40255`.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL `https://proceedings.mlr.press/v235/cai24b.html`.

Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL `https://arxiv.org/abs/2602.06036`. Accepted paper.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL `https://arxiv.org/abs/2107.03374`.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL `https://arxiv.org/abs/2110.14168`.

DeepSeek-AI. DeepSpec: A full-stack codebase for training and evaluating speculative decoding draft models. Official open-source software, 2026. URL `https://github.com/deepseek-ai/DeepSpec`.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. URL https://aclanthology.org/2024.acl-long.681/.

Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://proceedings.mlr.press/v235/fu24a.html.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems 34, Datasets and Benchmarks Track*, 2021. URL https://openreview.net/forum?id=7Bywt2mQsCe.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, 2019. URL https://proceedings.mlr.press/v97/houlsby19a.html.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022. URL https://www.microsoft.com/en-us/research/publication/lora-low-rank-adaptation-of-large-language-models/.

Shijing Hu, Jingyang Li, Xingyu Xie, Zhihui Lu, Kim-Chuan Toh, and Pan Zhou. GRIFFIN: Effective token alignment for faster speculative decoding. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/b6e67ae290635d0874c4cb43ba2a2cfb-Abstract-Conference.html.

Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. RepSpec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554c1f-Abstract-Conference.html.

Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/1b96f01343ff10150e6719eb163e1536-Abstract-Conference.html.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL https://doi.org/10.1145/3600006.3613165.

Haodi Lei, Yafu Li, Haoran Zhang, Shunkai Zhang, Qianjia Cheng, Xiaoye Qu, Ganqu Cui, Bowen Zhou, Ning Ding, Yun Luo, and Yu Cheng. Draft-OPD: On-policy distillation for speculative draft models. *arXiv preprint arXiv:2605.29343*, 2026. URL https://arxiv.org/abs/2605.29343.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, 2024a. URL https://proceedings.mlr.press/v235/li24bt.html.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024b. URL https://aclanthology.org/2024.emnlp-main.422/.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems 38*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/c7b5a35ea98b62512a869c19ea7b03cb-Abstract-Conference.html.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/aca97732e30bcf1303bc22ac3924fd16-Abstract-Conference.html.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024. URL https://arxiv.org/abs/2305.09781.

Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. OmniDraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3c2fe1417eed1c6ff9acf169617981ea-Abstract-Conference.html.

Damian Smith, Harvey Mannering, and Antonia Marcu. Functional alignment can mislead: Examining model stitching. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 55972–55998, 2025. URL https://proceedings.mlr.press/v267/smith25a.html.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise Parallel Decoding for Deep Autoregressive Models. In *Advances in Neural Information Processing Systems 31*, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/c4127b9194fe8562c64dc0f5bf2c93bc-Abstract.html.

Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Gaurav Jain, Oren Pereg, Moshe Wasserblat, and David Harel. Accelerating LLM inference with lossless speculative decoding algorithms for heterogeneous vocabularies. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025a. URL https://proceedings.mlr.press/v267/timor25a.html.

Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Oren Pereg, Moshe Wasserblat, Tomer Galanti, Michal Gordon-Kiwkowitz, and David Harel. Distributed speculative inference: Speculation parallelism for provably faster lossless language model inference. In *International Conference on Learning Representations*, 2025b. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/b36554b97da741b1c48c9de05c73993e-Abstract-Conference.html.

Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. OPT-Tree: Speculative Decoding with Adaptive Draft Tree Structure. *Transactions of the Association for Computational Linguistics*, 2025. URL https://aclanthology.org/2025.tacl-1.8/.

Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu Wei, and Zhifang Sui. Speculative Decoding: Exploiting Speculative Execution for Accelerating Seq2seq Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. URL https://aclanthology.org/2023.findings-emnlp.257/.

Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. URL https://aclanthology.org/2024.findings-acl.456/.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems 32*, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. URL https://aclanthology.org/2024.acl-long.607/.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 37*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

Xiandong Zou, Jianshu Li, Jing Huang, and Pan Zhou. Variational speculative decoding: Rethinking draft training from token likelihood to sequence acceptance. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL https://ink.library.smu.edu.sg/sis_research/11190/.

APPENDIX

The appendix gives implementation details (Appendix A), complete workload results (Appendix B), fitting studies (Appendix C), acceptance measurements (Appendix E), data checks (Appendix F) and fitting-time and memory measurements (Appendix G).

A IMPLEMENTATION AND MEASUREMENT DETAILS

Released models. The source is `Qwen/Qwen3-4B`. Targets are `Qwen/Qwen3-8B` and `Qwen/Qwen3-14B`. The source DFlash is `z-lab/Qwen3-4B-DFlash-b16`, and its native 8B control is `z-lab/Qwen3-8B-DFlash-b16`. EAGLE-3 uses DeepSpec's `eagle3_qwen3_4b_ttt7`, `eagle3_qwen3_8b_ttt7` and `eagle3_qwen3_14b_ttt7` releases. All model and upstream revisions are fixed in the accompanying configurations.

Features and dimensions. Layer indices count transformer blocks from zero. Source and Qwen3-8B states come from blocks $[1, 9, 17, 25, 33]$. Qwen3-14B states come from $[1, 10, 19, 28, 37]$. Joining five 4,096-wide states gives a 20,480-wide 8B input. Joining five 5,120-wide states gives a 25,600-wide 14B input. Both drafter interfaces have width 2,560. The matrix dimensions are therefore $2,560 \times 20,480$ and $2,560 \times 25,600$. The DFlash input normalization has no trainable gain or bias. The released output normalization remains frozen.

Training. Four workers average record losses at each update. Seed 1729 fixes the selected fit. AdamW uses learning rate 0.0006, zero weight decay and gradient clipping at 1.0. Computation uses BF16, with the feature loss computed in FP32. Each of 1,024 updates processes four records, each capped at 192 rendered tokens. The same map remains fixed across tasks.

Runtime and timing. The main AR runs use Python 3.12.13 and PyTorch 2.9.1 with CUDA 12.8. DFlash uses Transformers 5.3.0. EAGLE-3 uses the pinned 5.10.2 overlay. Each worker owns a full target replica on one L40S GPU. Four workers process independent requests rather than splitting one model over GPUs. One warmup per method precedes measured requests. Method order rotates across requests. The synchronized timer includes prompt processing, drafting, verification, feature updates and runtime bookkeeping. The historical block and memory experiments use RTX 6000 Ada GPUs.

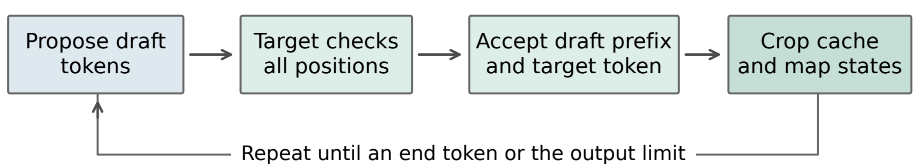

Figure 6: One generation cycle. RelaySpec changes the drafter's context. The target retains control over committed tokens and the shared decoder retains responsibility for positions, stopping and cache updates.

Stopping and conversation history. Generation stops at the first end token or the output cap. Capped requests remain in both timing and quality aggregates. Each MT-Bench method uses its own first-turn output as second-turn context. Two turns form one resampling unit. The breadth run has 750 initial records and 830 timed requests after expanding 80 conversations to two turns.

Aggregation. Each method's throughput is the sum of its output-token counts divided by the sum of its request times. Each of 10,000 resamples recomputes this ratio from matched requests, using bootstrap seed 1729. Confidence bounds are the central 95% of resampled estimates. The request-time ratio uses summed times alone. Table 5 reports both measurements and output lengths.

Table 5: Additional MATH-500 measurements. A is AR, S is source reuse and R is RelaySpec. Time ratio divides summed AR time by summed relay time.

Drafter	Target	Mean tokens AR / R	Time ratio AR / R	Throughput R / S	Progress S / R
DFlash	8B	783.22 / 778.81	5.018	1.531	7.66 / 6.98
DFlash	14B	777.66 / 772.33	5.143	1.268	7.44 / 6.49
EAGLE-3	8B	783.22 / 767.79	2.397	1.278	5.15 / 4.33
EAGLE-3	14B	777.66 / 766.20	2.625	1.097	5.07 / 4.01

B COMPLETE WORKLOAD RESULTS

Table 6: Complete DFlash breadth comparisons with paired AR. Ratios use end-to-end throughput. Every planned task and target cell is included.

Target	Task	Requests	AR tok/s	Relay tok/s	Relay / AR [95% CI]	Relay / source
8B	GSM8K	128	37.60	138.89	3.69 [3.59, 3.81]	1.456
8B	HumanEval	164	37.55	120.87	3.22 [3.14, 3.30]	1.244
8B	MBPP	378	37.49	129.35	3.45 [3.36, 3.54]	1.320
8B	MT-Bench	160	37.47	70.69	1.89 [1.76, 2.05]	1.387
14B	GSM8K	128	26.80	96.98	3.62 [3.52, 3.72]	1.104
14B	HumanEval	164	26.73	77.54	2.90 [2.83, 2.98]	0.890
14B	MBPP	378	26.72	83.82	3.14 [3.07, 3.21]	0.956
14B	MT-Bench	160	26.55	49.26	1.85 [1.73, 2.01]	1.086

The earlier fixed-map source comparisons in Table 7 provide all four workloads for both drafter families on RTX 6000 Ada. They are a separate paired experiment with their own denominator. The EAGLE-3 14B code cells show the reduced margin when lower accepted progress outweighs source-removal savings. This operating range complements the AR acceleration measured in the main comparison.

Table 7: Historical paired source-reuse breadth measurements on RTX 6000 Ada. Ratios compare relay with source throughput within these runs. They are not normalized by an AR arm from another experiment.

Drafter	Target	Task	Requests	Source tok/s	Relay tok/s	Ratio [95% interval]
DFlash	8B	GSM8K	128	116.87	166.39	1.4237 [1.4040, 1.4445]
DFlash	8B	HumanEval	164	118.84	145.00	1.2201 [1.2017, 1.2383]
DFlash	8B	MBPP	378	117.77	151.55	1.2868 [1.2732, 1.3001]
DFlash	8B	MT-Bench	160	60.85	81.62	1.3412 [1.3183, 1.3639]
DFlash	14B	GSM8K	128	88.04	97.07	1.1025 [1.0836, 1.1214]
DFlash	14B	HumanEval	164	87.34	77.61	0.8886 [0.8741, 0.9038]
DFlash	14B	MBPP	378	88.02	83.98	0.9541 [0.9416, 0.9663]
DFlash	14B	MT-Bench	160	45.32	49.30	1.0877 [1.0653, 1.1083]
EAGLE-3	8B	GSM8K	128	85.99	108.45	1.2613 [1.2437, 1.2789]
EAGLE-3	8B	HumanEval	164	71.75	81.10	1.1304 [1.1172, 1.1434]
EAGLE-3	8B	MBPP	378	69.85	82.10	1.1753 [1.1656, 1.1850]
EAGLE-3	8B	MT-Bench	160	42.27	49.45	1.1698 [1.1501, 1.1906]
EAGLE-3	14B	GSM8K	128	67.81	73.41	1.0826 [1.0675, 1.0979]
EAGLE-3	14B	HumanEval	164	55.38	50.58	0.9132 [0.9011, 0.9255]
EAGLE-3	14B	MBPP	378	54.55	50.49	0.9256 [0.9151, 0.9361]
EAGLE-3	14B	MT-Bench	160	33.83	34.57	1.0221 [0.9999, 1.0449]

Table 8: Task scores for the historical fixed-map breadth runs. Source reuse and RelaySpec share each reported score. Code columns are single-program pass percentages under base and expanded EvalPlus tests.

Drafter	Target	GSM8K	HumanEval	HumanEval+	MBPP	MBPP+
DFlash	8B	93.75	85.98	80.49	84.13	73.02
DFlash	14B	94.53	89.02	83.54	88.36	75.13
EAGLE-3	8B	92.97	87.20	82.32	83.07	71.96
EAGLE-3	14B	94.53	90.24	85.98	88.89	74.87

Across these historical math/code comparisons, all 4,680 paired task outcomes agree between source reuse and RelaySpec. This count is four model/drafter pairs times $500 + 128 + 164 + 378 = 1,170$ problems. Expanded code tests evaluate the same programs and do not add independent examples. For the current AR-paired DFlash GSM8K runs, the scorer gives 120/128 correct for all arms at 8B and 121/128 at 14B. Code scores above refer specifically to the historical outputs, preserving scorer/run identity.

AR-paired EAGLE-3 code quality. We additionally score the saved programs from the EAGLE-3 AR-paired breadth runs with official EvalPlus base and expanded tests. Table 9 compares one program per task against AR from the same run. At 8B, RelaySpec passes four fewer MBPP tasks under both suites. At 14B, it passes two more. HumanEval expanded-test totals match AR at both target sizes, although individual task outcomes differ. RelaySpec and source reuse have identical per-task outcomes in all eight cells. The paired intervals cover zero. This does not establish equality or noninferiority. These retrospective comparisons use one fitted map per setting and do not include fitting-seed uncertainty. Conversation quality remains unmeasured.

Table 9: EAGLE-3 AR-paired code quality. Counts are passed tasks. The intervals resample paired tasks. Expanded tests reuse the same programs and do not add independent samples.

Target/task	Suite	AR	Relay	Δ (pp)	Paired 95% interval
8B HumanEval	base	143/164	141/164	-1.22	$[-3.66, +1.22]$
8B HumanEval	plus	133/164	133/164	+0.00	$[-3.05, +3.05]$
8B MBPP	base	318/378	314/378	-1.06	$[-2.91, +0.79]$
8B MBPP	plus	276/378	272/378	-1.06	$[-2.65, +0.53]$
14B HumanEval	base	143/164	143/164	+0.00	$[-2.44, +2.44]$
14B HumanEval	plus	134/164	134/164	+0.00	$[-2.44, +2.44]$
14B MBPP	base	338/378	340/378	+0.53	$[-1.32, +2.38]$
14B MBPP	plus	283/378	285/378	+0.53	$[-1.06, +2.12]$

C Fitting and interface studies

Table 10: DFlash-8B block-size ablation on 64 questions, a 512-token cap and RTX 6000 Ada hardware. AR is measured in each run.

Block	AR tok/s	Native draft tok/s	Relay tok/s	Relay / AR	Progress / cycle
8	44.68	175.12	164.90	3.691	4.98
16	44.67	229.87	210.15	4.704	6.48
32	44.76	207.70	131.36	2.935	4.14

Distinct records at fixed fitting work. A separate controlled DFlash-8B study holds fitting at 1,024 updates and four records per update while varying distinct records from 64 to 4,096. Smaller sets repeat in the same ordered schedule. Table 11 reports end-to-end throughput on the same 128 development-exposed MATH questions. Of the measured settings, 512 is the smallest within 5% of the best observed mapper throughput. The 256-record setting retains 91.4% of that observed best. This is a descriptive, single-seed boundary, not a minimum across settings or a confirmed optimum.

The 512–4,096 settings form an unresolved plateau. Selecting the reference from the same evaluations does not provide independent confirmation. Every relay scores 105/128, versus AR's 104/128.

Table 11: Distinct-data study at 4,096 presentations for every row. Native retention uses each run's paired native-drafter control. Best retention uses the observed best relay across these settings. All results condition on one fitting seed.

Records	Tokens/s	$\times$ AR	Native retained	Best retained	Correct
64	134.39	3.55	65.7%	72.8%	105/128
128	154.56	4.10	75.5%	83.7%	105/128
256	168.76	4.51	82.6%	91.4%	105/128
512	181.91	4.84	88.9%	98.5%	105/128
1,024	184.55	4.92	90.3%	99.9%	105/128
2,048	183.81	4.88	89.8%	99.5%	105/128
4,096	184.64	4.90	90.2%	100.0%	105/128

Continuous optimization. We also save checkpoints from one uninterrupted 4,096-record fit at 128, 256, 512, 1,024 and 2,048 updates. They have seen respectively 512, 1,024, 2,048, 4,096 and 4,096 distinct records. The final point therefore measures a second pass, not 8,192 distinct examples. End-to-end AR-relative throughput increases from 3.65 to 5.03 across this trajectory. All five checkpoints score 105/128. Unlike the earlier separately fitted budget configurations, these checkpoints share one optimization trajectory. Figure 4 separates this experiment from the fixed-work data study.

Table 12: Completed DFlash-8B fitting studies on the same 128-question set. Records count distinct fitting examples. Two passes over 4,096 records give 8,192 presentations. Source reuse is paired within each run.

Fitting choice	Records	Updates	Relay tok/s	Relay / source [95% CI]	Score
512 records	512	128	136.78	1.150 [1.118, 1.181]	82.03
1,024 records	1,024	256	162.04	1.372 [1.347, 1.397]	82.03
2,048 records	2,048	512	173.72	1.477 [1.458, 1.495]	82.03
4,096 records, two passes	4,096	2,048	187.85	1.573 [1.558, 1.588]	82.03
Ridge surrogate	4,096	*solve*	115.18	0.955 [0.927, 0.982]	82.03
Nonlinear width 512	4,096	1,024	104.49	0.872 [0.841, 0.903]	82.03

The data-budget configurations use the same normalized linear interface and relative-error objective. The nonlinear map uses an intermediate width of 512 and about 11.8M parameters, compared with 52.4M for the dense 8B map. Its lower throughput is consistent with a less effective fitted interface, but the comparison also changes capacity. It does not isolate a disadvantage of nonlinear functions. The ridge configuration solves a raw linear surrogate with ridge coefficient 1.0. Since the deployed DFlash interface includes output normalization, this is a different objective from Equation 4.

Matched capacity on a separate fitting source. We additionally fit 30 DFlash-8B configurations on nested sets of 512 and 2,048 NuminaMath-CoT records,[1] using a fixed 1,024-record fitting validation split. The source-stratified sets include `cn_k12`, `orca_math` and `synthetic_math` examples. Their lexical filter excludes exact normalized problem/solution matches and near matches at five-shingle Jaccard similarity at least 0.6 for texts with at least ten unique shingles. This filter does not establish semantic or template independence. These results use a separate fitting source from the historical MATH study above.

Each configuration receives 8,192 updates with four records per update: 64 passes at $N = 512$ and 16 at $N = 2048$. Frozen features are cached once. Factored linear maps and bias-free two-layer GELU MLPs use widths 64, 128, 256, 512, 1,024, 2,048 and 4,096, giving matched parameter counts $h(20{,}480 + 2{,}560)$. The dense reference has 52.43M parameters. The rank of a linear function

[1] `AI-MO/NuminaMath-CoT`, revision `9d8d210c9f6a`, training shard 0 of 5.

remains bounded by 2,560 even when its factorization width is larger. All cells share the data order, relative-interface objective, learning rate 6×10^{-4} and seed 1729.

Table 13: Validation relative interface MSE for the capacity study. Linear and MLP entries at the same width have equal parameter counts. The dense row supplies the single-linear-layer reference.

Width	Params (M)	Linear, N=512	MLP, N=512	Linear, N=2048	MLP, N=2048
64	1.47	0.5732	0.5790	0.5692	0.5752
128	2.95	0.4761	0.4871	0.4690	0.4804
256	5.90	0.3760	0.3883	0.3648	0.3767
512	11.80	0.2862	0.3082	0.2685	0.2878
1024	23.59	0.2295	0.2579	0.2040	0.2224
2048	47.19	0.2133	0.2438	0.1847	0.1982
4096	94.37	0.2145	0.2501	0.1845	0.1936
Dense	52.43	0.2101	—	0.1805	—

At this optimization budget, wider factorizations approach the dense reference, while MLPs have higher validation error at every matched width. Increasing MLP width from 2,048 to 4,096 at $N = 512$ lowers diagnostic training error from 0.1586 to 0.1278 but raises validation error from 0.2438 to 0.2501. At $N = 2048$, validation error instead decreases from 0.1982 to 0.1936. This interaction motivates checking regularization and convergence before attributing the ranking to function class.

More epochs on fixed smaller datasets. We continue the width-512 linear and MLP fits to 32,768 total updates, preserving Adam moments, random states and cyclic data position. A separate bounded pilot reproduced uninterrupted fitting bit for bit. Figure 7 shows late-training diagnostics on the same first 256 training records and all 1,024 validation records. The $N = 2048$ MLP improves from 0.2878 to 0.2787 validation error between 8,192 and 32,768 updates. At $N = 512$, linear training error falls from 0.2443 to 0.2350 while validation error rises from 0.2862 to 0.2873. The latter is consistent with mild overfitting on this diagnostic split. These single-seed fitting trajectories require separate decoding and quality evaluation, including replication across targets and workloads.

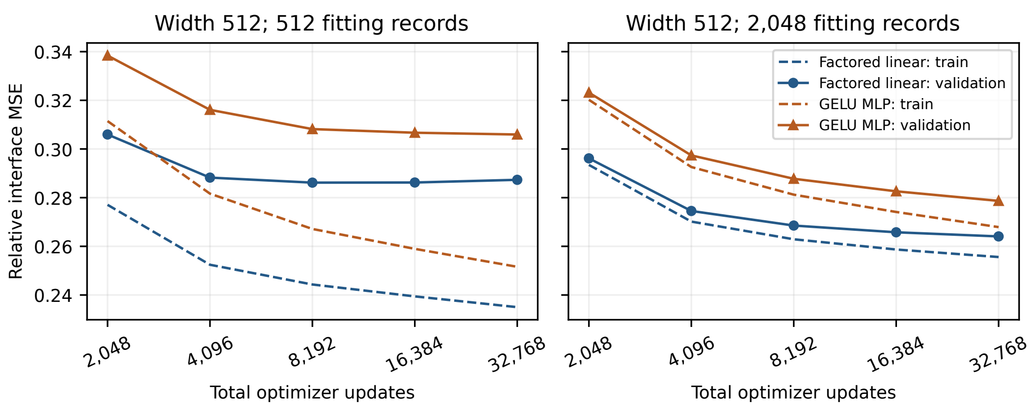

Figure 7: Late-training trajectories, starting at 2,048 updates. Dashed lines use the fixed training diagnostic subset. Solid lines use fitting validation. The horizontal axis counts updates, while the distinct datasets remain fixed at 512 or 2,048 records.

Explicit regularization on fixed smaller datasets. We complete 42 nonzero-penalty fits for the dense, width-512 factored linear and width-512 GELU MLP connectors at $N \in \{512, 2048\}$. Each fit uses 8,192 updates, the same 1,024 validation records and seed 1729. Explicit L2 coefficients are $10^{-7}, 10^{-6}, 10^{-5}, 10^{-4}$. Separate AdamW arms use decay $10^{-4}, 10^{-3}, 10^{-2}$. No tested L2 coefficient improves its matched zero-penalty endpoint. The largest AdamW improvement is a 0.17% relative decrease in validation error for the width-512 linear connector at $N = 512$. Neither MLP setting improves with either penalty family. Table 14 reports the best validation value within each penalty family, so these minima are development selections. Figure 8 retains all tested coefficients. This grid does not establish a decoding gain or a penalty effect across seeds, and it does not resolve regularization for the wider MLPs.

Table 14: Matched regularization endpoints. Lower relative interface MSE is better. Each nonzero minimum is selected from four L2 or three AdamW coefficients using the same fitting validation split.

N	Mapper	No penalty	Best tested L2	Best tested AdamW
512	Dense	0.21008	0.21138	0.21011
512	Factored linear 512	0.28617	0.28738	0.28570
512	GELU MLP 512	0.30818	0.30827	0.30822
2048	Dense	0.18055	0.18424	0.18055
2048	Factored linear 512	0.26853	0.26970	0.26852
2048	GELU MLP 512	0.28776	0.28864	0.28779

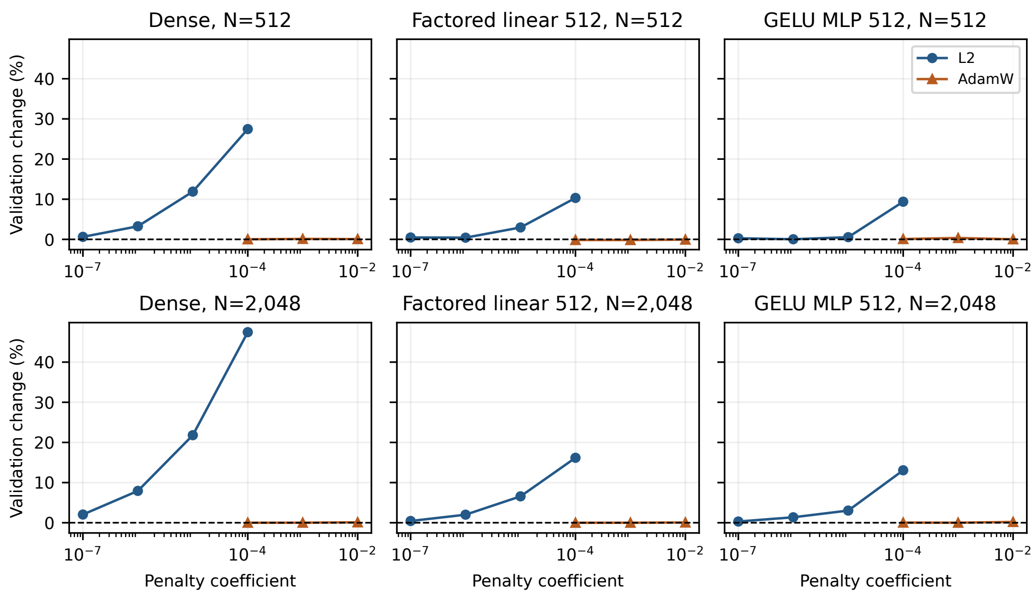

Figure 8: All 42 penalty endpoints relative to their matched zero-penalty validation error. Positive values indicate worse fitting validation. The two curves are separate objective-L2 and AdamW experiments.

Table 15: Matched EAGLE-3 8B normalization comparison on 32 development questions after 128 updates. Throughput ratios use source reuse.

Input to linear map	Relative error	Throughput ratio [95% interval]	Progress retained
Normalized features	0.413	1.035 [1.003, 1.068]	69.0%
Raw features	0.266	1.237 [1.213, 1.263]	82.6%

Table 16: DFlash objective comparison on 32 development questions. Candidates share fitting data, update count, normalization and optimizer. Each ratio uses the source-reuse arm of that candidate's paired run.

Target	Objective	Throughput ratio [95% interval]	Token matches
8B	Relative error	1.523 [1.488, 1.558]	32/32
8B	Squared error + cosine	1.509 [1.476, 1.545]	32/32
14B	Relative error	1.265 [1.243, 1.289]	32/32
14B	Squared error + cosine	1.251 [1.229, 1.276]	32/32

Relative error weights a record's feature mismatch by teacher magnitude. Squared error plus 0.1 times cosine distance combines coordinate error with directional agreement. Direct candidate-to-candidate throughput ratios are 1.0085 [1.0005, 1.0174] at 8B and 1.0114 [1.0046, 1.0177] at 14B. These are modest gains with one fewer mixing coefficient.

C.1 EAGLE-3 FITTING REPLICATION

We repeat the fixed smaller-data fitting study with the EAGLE-3 source drafter and the 8B target. The grid contains dense, factored linear and GELU MLP connectors, with widths 128, 512 and 2048 for the latter two. Each primary fit uses 8,192 updates at $N \in \{512, 2048\}$ and the same 1,024 fitting-validation records. Two additional seeds repeat the dense $N = 2048$ setting. This experiment uses the EAGLE input convention without the DFlash input normalization. Error values are interpreted within each drafter family's conditioning interface.

Table 17: EAGLE-3 fitting validation. End denotes the 8,192-update endpoint. Best denotes the minimum saved fitting-validation error, with its update count. These are development selections for seed 1729, not decoding results. M weights counts millions of trainable mapper parameters.

		$N = 512$			$N = 2048$		
Mapper	M weights	End	Best	Update	End	Best	Update
Dense	52.43	0.24787	0.21666	1024	0.21203	0.21061	2048
Linear 128	2.95	0.41980	0.41980	8192	0.41505	0.41505	8192
Linear 512	11.80	0.28632	0.28505	2048	0.27188	0.27188	8192
Linear 2048	47.19	0.23486	0.22877	2048	0.21132	0.21132	8192
MLP 128	2.95	0.46554	0.46554	8192	0.45645	0.45645	8192
MLP 512	11.80	0.31749	0.31749	8192	0.29577	0.29577	8192
MLP 2048	47.19	0.28154	0.27124	2048	0.23196	0.22683	4096

The dense $N = 512$ mapper reaches 0.21666 at 1,024 updates, then rises to 0.24787 at the final update. Its endpoint is worse than the width-2048 linear mapper, but its best saved validation value is better than that linear mapper's 0.22877. At $N = 2048$, the dense and width-2048 linear minima are 0.21061 and 0.21132. Thus endpoint rankings alone can reflect different convergence trajectories. At every matched width in this grid, the MLP's best saved validation error exceeds the linear mapper's minimum. The three dense $N = 2048$ endpoints range from 0.212034 to 0.212043. This narrow seed spread concerns one fitting setting. It does not establish seed stability for the other architectures or for decoding performance.

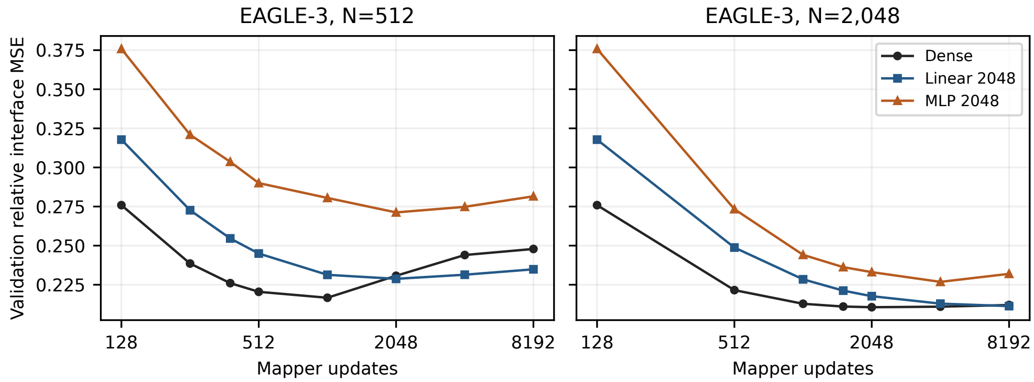

Figure 9: EAGLE-3 fitting-validation trajectories at fixed distinct-data counts. Additional updates can worsen validation, most visibly for the dense mapper at $N = 512$. The horizontal axis counts updates rather than new examples. The subsequent checkpoint comparison measures actual decoding.

C.2 Feature-validation minima and decoding

We compare each distinct saved feature-validation minimum with its 8,192-update endpoint in two declared decoding campaigns. Each uses 16 paired development questions, greedy decoding and a 256-token cap. Checkpoint selection precedes decoding. Table 18 includes every distinct early-versus-endpoint pair in these campaigns.

Table 18: Decoding throughput at the minimum feature-validation checkpoint relative to the same fit's endpoint. Intervals resample paired requests 10,000 times. They are descriptive, without multiple-comparison adjustment, and do not measure variation across fitting seeds.

Family	Mapper	N	Learning rate	Selected update	Early / end [95% CI]
DFlash	Linear 4096	512	0.0006	4096	1.009 [0.981, 1.040]
DFlash	MLP 4096	512	0.0006	4096	0.978 [0.961, 0.998]
DFlash	Linear 4096	512	0.0002	4096	1.008 [0.985, 1.029]
DFlash	Linear 4096	512	0.0018	4096	1.003 [0.978, 1.029]
DFlash	MLP 4096	512	0.0002	2048	1.011 [0.986, 1.039]
DFlash	MLP 4096	512	0.0018	4096	0.961 [0.939, 0.983]
EAGLE-3	Dense	512	0.0006	1024	0.950 [0.924, 0.976]
EAGLE-3	Linear 512	512	0.0006	2048	0.969 [0.948, 0.988]
EAGLE-3	Linear 2048	512	0.0006	2048	0.964 [0.922, 1.003]
EAGLE-3	MLP 2048	512	0.0006	2048	0.968 [0.942, 0.996]
EAGLE-3	Dense	2048	0.0006	2048	0.980 [0.969, 0.992]
EAGLE-3	MLP 2048	2048	0.0006	4096	1.019 [0.995, 1.046]

Lower feature-validation loss does not consistently predict faster decoding. For EAGLE-3's dense $N = 512$ fit, the selected 1,024-update checkpoint retains 94.99% of endpoint throughput, with interval [92.39, 97.59]%. For the DFlash width-4096 MLP at $N = 512$ and learning rate 0.0006, the selected 4,096-update checkpoint retains 97.77%, with interval [96.09, 99.76]%. Other intervals cross equal throughput. These results support treating feature regression as a surrogate and measuring decoding when selecting a connector. They do not establish full-answer quality or justify selection on untouched evaluation data.

Table 19: Observed throughput ranges across three fitting seeds in a separate 16-question, 256-token development campaign. Selected uses each fit's minimum saved feature-validation checkpoint. These ranges describe the three measured fits and runtime variation, not confidence intervals.

Mapper	N	Learning rate	Endpoint tok/s range	Selected tok/s range
Dense	512	0.0006	184.34–184.68	183.36–184.58
Dense	2048	0.0006	186.11–189.22	186.11–189.22
Linear 1024	512	0.0006	179.47–181.23	179.47–181.23
Linear 4096	2048	0.0002	185.36–187.81	185.36–187.81
MLP 4096	2048	0.0002	183.92–184.78	183.92–184.78
MLP 4096	512	0.0002	174.53–177.30	175.57–177.56

All six settings in Table 19 use the same cached features and seeds 1729, 1730 and 1731. The dense $N = 512$ endpoints span 184.34–184.68 tokens/s. The width-4096 $N = 2048$ MLP improves on its $N = 512$ counterpart, while the dense connector remains competitive. These short development runs do not establish the minimum sufficient data count or distinguish small architecture gaps from runtime variation.

C.3 Longer development outputs for small-data mappers

We evaluate nine saved DFlash connectors on 128 paired MATH development questions with a 2,048-token cap, greedy decoding and the same source, target and native-drafter references. These exposed questions provide a larger development comparison than the 16-question, 256-token screen. They are separate from any untouched confirmation set. The continued width-512 fits reuse the same 2,048 distinct training examples.

Table 20: Capped development outcomes for small-data DFlash fitting. Throughput retention uses the dense $N = 2048$, 8,192-update connector. Intervals use 10,000 paired request bootstrap samples. Correct counts scored answers. Cap counts outputs reaching 2,048 tokens, which can include EOS exactly at the limit and are not exact truncation counts.

Method	N	Updates	Tok/s	Dense retained (%) [95% CI]	Correct	Cap
AR	–	–	37.72	19.04 [18.23, 19.90]	104/128	11
Native drafter	–	–	204.57	103.27 [102.35, 104.17]	105/128	9
Source reuse	–	–	120.96	61.06 [60.53, 61.59]	105/128	9
Dense	512	8192	193.68	97.78 [97.09, 98.47]	105/128	9
Dense	2048	8192	198.09	100.00 [100.00, 100.00]	105/128	9
Linear 1024	512	8192	188.19	95.00 [94.14, 95.85]	105/128	9
Linear 4096	2048	8192	196.25	99.07 [98.50, 99.60]	105/128	9
MLP 4096	2048	8192	190.15	95.99 [95.30, 96.70]	105/128	9
Linear 512	2048	8192	164.61	83.10 [81.81, 84.38]	105/128	9
Linear 512	2048	32768	167.29	84.46 [83.15, 85.71]	105/128	9
MLP 512	2048	8192	154.39	77.94 [76.50, 79.35]	105/128	9
MLP 512	2048	32768	157.68	79.60 [78.07, 81.06]	105/128	9

Dense fitting on 512 examples retains 97.78% of the dense $N = 2048$ throughput, with interval [97.09, 98.47]%. This establishes a lower-data performance point in this setting, but does not establish that 512 is minimal. The width-1024 linear mapper at $N = 512$ retains 95.00%, with interval [94.14, 95.85]%, leaving its 95% boundary unresolved. The width-4096 linear mapper is slightly below the dense connector in this larger sample, so the earlier short-screen point ranking should not be interpreted as an established architecture advantage.

All nine connectors score 105 of 128 answers, compared with 104 for AR. Their paired accuracy-difference intervals are $[-0.03125, 0.046875]$. These intervals do not establish a one-percentage-point noninferiority margin. Nine connector outputs and eleven AR outputs reach the token cap. The observed scores therefore concern the configured capped evaluation, not uncapped answer quality. Longer width-512 fitting gives modest throughput increases while remaining below the wider or dense connectors. This comparison does not treat additional updates as additional data.

C.4 EAGLE-3 LONGER-OUTPUT CAPACITY COMPARISON

Two disjoint 64-question shards evaluate eight saved EAGLE-3 maps and three shared controls on the same 128 exposed MATH development questions. All 1,408 expected method/request pairs are present. The 2,048-token cap, fitting checkpoints, source revisions and pinned scorer remain fixed across shards. An eight-request pilot precedes this evaluation.

Table 21: EAGLE-3 with an 8B target on 128 exposed development questions. Dense retention uses the $N = 2048$ endpoint. Intervals are paired 95% request-bootstrap intervals conditional on the fits. Cap counts all outputs reaching 2,048 tokens. It is not an exact truncation count.

Method	N	Tokens/s	Dense retained (%) [95% CI]	Correct	Cap
AR	–	36.32	38.19 [37.14, 39.25]	104/128	11
Native drafter	–	94.89	99.78 [98.86, 100.80]	105/128	7
Source reuse	–	66.80	70.24 [69.69, 70.84]	105/128	7
Dense	512	93.47	98.28 [97.59, 99.01]	105/128	7
Dense	2048	95.10	100.00 [100.00, 100.00]	105/128	7
Linear 2048	512	93.85	98.69 [98.08, 99.31]	105/128	7
Linear 2048	2048	93.59	98.41 [97.76, 99.12]	105/128	7
MLP 2048	512	84.67	89.03 [88.24, 89.87]	105/128	7
MLP 2048	2048	89.25	93.85 [93.27, 94.46]	105/128	7
Linear 512	2048	84.91	89.29 [88.54, 90.09]	105/128	7
MLP 512	2048	79.76	83.87 [82.94, 84.84]	105/128	7

Dense $N = 512$ retains 98.28% of dense $N = 2048$ throughput. Linear-2048 at $N = 512$ retains 98.69%, while the two MLP-2048 fits retain 89.03% and 93.85% at $N = 512$ and $N = 2048$. The width-512 endpoints are slower. Every speculative method scores 105/128 with seven cap-length outputs. AR scores 104/128 with eleven. All speculative methods have identical per-question correctness in this sample, but the conservative paired 95% difference interval is still $[-3.37, 3.37]$ percentage points. Against AR, dense $N = 2048$ has four beneficial and three adverse correctness discordances. Equal or close scores therefore do not establish tight noninferiority or uncapped answer quality. The observed small-data retention replicates across the two drafter families under these development conditions, not across independent benchmark samples.

C.5 INPUT-ONLY TASK COMPLEXITY

We stratify the same 128 development requests using the benchmark's original difficulty levels and subjects, together with shared tokenized input length. Group definitions are fixed before computing subgroup outcomes. They do not use candidate success, generated output length or latency. This is a retrospective analysis of exposed development data, not a new confirmatory test. All twelve methods and all requests remain in the analysis.

Table 22: Throughput retained from dense $N = 2048$ within each MATH difficulty group, in percent with descriptive paired 95% intervals. All fits use 8,192 updates except the continued width-512 fits, which use 32,768 updates on the same 2,048 examples. For fitted maps, unspecified N is 2,048. Intervals do not adjust for multiple subgroup comparisons or model selection.

Setting	Levels 1–2 ($n = 42$)	Level 3 ($n = 21$)	Levels 4–5 ($n = 65$)
AR	20.1 [18.6, 21.5]	19.0 [16.5, 22.1]	18.7 [17.8, 19.7]
Native drafter	101.4 [99.1, 104.1]	104.2 [102.0, 106.6]	103.7 [102.8, 104.6]
Source reuse	60.6 [59.2, 61.8]	60.7 [59.2, 62.1]	61.3 [60.6, 61.9]
Dense, $N = 512$	97.4 [96.1, 98.6]	97.9 [96.1, 100.2]	97.9 [97.0, 98.8]
Dense, $N = 2048$	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Linear 1024, $N = 512$	94.5 [93.2, 95.9]	94.8 [93.4, 96.5]	95.2 [94.0, 96.4]
Linear 4096, $N = 2048$	98.9 [97.9, 99.8]	100.0 [98.6, 101.5]	98.9 [98.2, 99.6]
MLP 4096, $N = 2048$	95.5 [94.3, 96.6]	96.8 [94.9, 98.7]	96.0 [95.1, 96.9]
Linear 512	82.6 [79.3, 85.7]	84.5 [81.6, 86.8]	83.0 [81.5, 84.5]
Linear 512, continued	84.1 [80.6, 87.5]	86.2 [84.0, 88.2]	84.2 [82.7, 85.7]
MLP 512	76.5 [73.4, 79.5]	79.9 [76.8, 82.5]	78.0 [76.2, 79.7]
MLP 512, continued	78.4 [74.6, 82.0]	79.8 [77.0, 82.0]	80.0 [78.1, 81.7]

Dense $N = 512$ retains 97.4–97.9% of dense $N = 2048$ throughput across the three difficulty groups. The width-1024 linear map stays near the 95% boundary, while MLP-4096 retains 95.4–96.8%. The smaller width-512 maps remain slower in every group. These results describe the tested capacities and workloads. They do not establish an optimal size across settings.

For levels 1–2, level 3 and levels 4–5, respectively, AR answers 38/42, 19/21 and 47/65 questions correctly. Every mapper answers 38/42, 18/21 and 49/65 correctly. All eleven AR cap hits occur in the highest difficulty group, as do eight of the nine mapper cap hits. Difficulty therefore coincides with changes in capped answer quality and output length. It is not a controlled intervention on decoding cost.

Input-length bins of at most 64, 65–128, 129–256 and more than 256 tokens contain 52, 57, 15 and 4 requests. In the final four-request group, dense $N = 512$ retains 94.5% [91.3, 96.4]% and linear-1024 retains 90.8% [88.2, 94.4]% of dense $N = 2048$ throughput. The small sample and multiple comparisons make this a signal for replication, not evidence of a causal length effect or a new tuning rule. All seven original subject strata, input assignments, method outcomes and conservative paired accuracy intervals are retained in the accompanying artifacts. No additional generation or fitting is used for this analysis.

C.6 Capacity replication with a 14B target

We repeat the fixed-data capacity comparison with Qwen3-14B for both DFlash and EAGLE-3. Each family has fourteen primary fits: dense, factored linear and GELU MLP widths 512, 1,024 and 4,096 at each of $N = 512$ and $N = 2048$. Two additional dense fits use seed 1730. All receive 8,192 batch-four updates at learning rate 6×10^{-4}, with no L2 penalty or weight decay. Validation uses the same 1,024 records within each family. Training diagnostics use a 256-record prefix. The five target taps give 25,600 input dimensions and the output has 2,560 dimensions. Dense therefore has 65.54M parameters, versus 14.42M, 28.84M and 115.34M at the three matched factorized/MLP widths. Width 4,096 does not increase the maximum rank of a linear function beyond the output dimension.

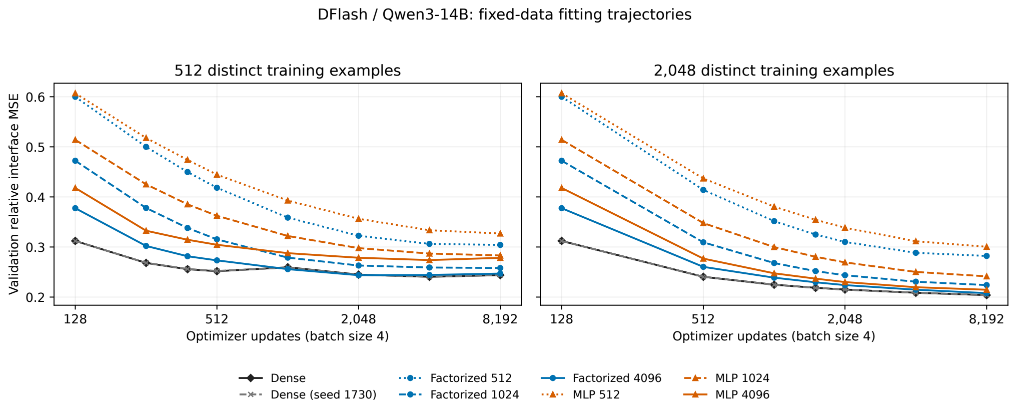

Figure 10: DFlash with Qwen3-14B. Validation error across the saved training trajectory, with all capacities and both dense seeds retained. The decoding endpoint remains fixed at 8,192 updates.

Table 23: DFlash-14B fixed endpoints. Throughput retention uses dense $N = 2048$, seed 1729. Intervals are paired 95% request-bootstrap intervals on sixteen exposed questions capped at 256 output tokens. They exclude fitting-seed and selection uncertainty.

Mapper	N	Params (M)	Train / val.	Tokens/s	Retained (%)
Dense	512	65.54	0.153 / 0.244	123.38	96.0 [92.3, 99.3]
Dense (seed 1730)	512	65.54	0.153 / 0.244	122.08	94.9 [91.5, 98.1]
Linear 512	512	14.42	0.251 / 0.304	99.16	77.1 [72.6, 81.8]
Linear 1024	512	28.84	0.180 / 0.258	117.39	91.3 [87.7, 95.1]
Linear 4096	512	115.34	0.155 / 0.247	121.65	94.6 [91.4, 97.9]
MLP 512	512	14.42	0.277 / 0.327	92.11	71.6 [68.1, 75.8]
MLP 1024	512	28.84	0.208 / 0.283	109.02	84.8 [79.7, 90.3]
MLP 4096	512	115.34	0.140 / 0.278	112.85	87.8 [83.5, 92.0]
Dense	2048	65.54	0.188 / 0.204	128.58	100.0 [100.0, 100.0]
Dense (seed 1730)	2048	65.54	0.188 / 0.204	129.55	100.8 [99.5, 102.1]
Linear 512	2048	14.42	0.274 / 0.282	105.53	82.1 [76.9, 87.4]
Linear 1024	2048	28.84	0.212 / 0.224	127.67	99.3 [96.5, 102.3]
Linear 4096	2048	115.34	0.193 / 0.208	126.96	98.7 [96.5, 101.2]
MLP 512	2048	14.42	0.292 / 0.300	96.46	75.0 [70.9, 79.2]
MLP 1024	2048	28.84	0.227 / 0.241	121.21	94.3 [90.9, 98.1]
MLP 4096	2048	115.34	0.174 / 0.215	122.32	95.1 [92.6, 98.2]

With DFlash, the wide MLP at $N = 2048$ has lower training error than dense, but higher validation error and lower measured throughput. The width-1024 linear map nearly matches dense throughput with fewer parameters. These comparisons use fixed endpoints, not checkpoints chosen after inspecting decoding. Every method reaches the output cap on these sixteen questions, so the short scores do not establish full-answer quality. The runtime includes AR and source-reuse controls. No native 14B DFlash checkpoint is substituted from another target.

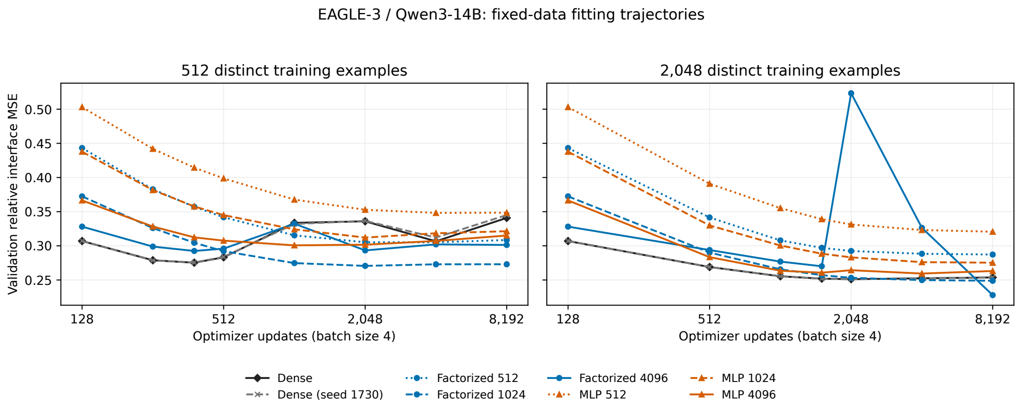

Figure 11: EAGLE-3 with Qwen3-14B under the same data-count and update budgets. The width-4096 linear trajectory retains its intermediate validation spike. Curves show optimization behavior, not a ranking of decoding performance or a selected stopping rule.

Table 24: EAGLE-3-14B fixed endpoints under its own runtime. Retention and intervals use the same within-family protocol as Table 23. Absolute throughput across the two tables is not an algorithm-only comparison.

Mapper	N	Params (M)	Train / val.	Tokens/s	Retained (%)
Dense	512	65.54	0.234 / 0.341	62.24	94.4 [91.7, 97.2]
Dense (seed 1730)	512	65.54	0.238 / 0.345	63.32	96.1 [92.7, 99.5]
Linear 512	512	14.42	0.256 / 0.308	56.92	86.4 [83.5, 89.4]
Linear 1024	512	28.84	0.197 / 0.273	62.50	94.8 [91.9, 98.0]
Linear 4096	512	115.34	0.226 / 0.301	60.90	92.4 [89.8, 95.1]
MLP 512	512	14.42	0.302 / 0.349	53.15	80.6 [77.4, 84.1]
MLP 1024	512	28.84	0.253 / 0.321	59.45	90.2 [87.3, 92.8]
MLP 4096	512	115.34	0.224 / 0.315	52.64	79.9 [76.6, 83.5]
Dense	2048	65.54	0.231 / 0.254	65.91	100.0 [100.0, 100.0]
Dense (seed 1730)	2048	65.54	0.231 / 0.254	66.45	100.8 [99.3, 102.4]
Linear 512	2048	14.42	0.279 / 0.287	58.92	89.4 [86.0, 92.7]
Linear 1024	2048	28.84	0.236 / 0.249	63.41	96.2 [94.6, 97.8]
Linear 4096	2048	115.34	0.216 / 0.228	65.61	99.5 [97.5, 101.9]
MLP 512	2048	14.42	0.311 / 0.321	56.54	85.8 [82.2, 89.3]
MLP 1024	2048	28.84	0.259 / 0.275	61.76	93.7 [90.5, 97.0]
MLP 4096	2048	115.34	0.227 / 0.263	64.02	97.1 [94.9, 99.4]

EAGLE-3 differs from DFlash in its fitting dynamics. At $N = 512$, both dense seeds have their lowest saved validation error at 384 updates, followed by substantial increases. At $N = 2048$, the wide linear endpoint has lower validation error than dense, but nearly equal decoding throughput. Its intermediate spike also shows that the shared learning rate does not produce uniformly smooth training. These observations motivate matched learning-rate checks before attributing a difference to the mapper function class. Four lower-rate 16-update pilots passed numerical and decoding checks, but their proper 8,192-update fits were not executed within the experiment timebox. The pilots do not establish whether a lower rate resolves the degradation. Lower feature error alone does not establish a decoding gain. Full-answer quality, broader workloads and additional fitting seeds remain separate from this capped comparison.

C.7 Matched warm-training budgets on 512 examples

We compare feature regression, connector cross-entropy (CE) and rank-32 drafter LoRA on the same 512-example pool. CE updates the connector while LoRA holds the initial connector fixed. Both use target-greedy teacher labels on the last 15 proposal positions. An equal three-rate development screen selects learning rate 2×10^{-5} for each adaptation family. Feature regression uses its declared rate 6×10^{-4}. The two LoRA seeds vary the update initialization and share the same initial mapper. They do not measure variation across initial mapper fits.

We measure 91.413 seconds of optimizer-update time for the 8,192-update feature baseline and declare budgets of 0.25, 1 and 4 times that cost. CE and LoRA begin with a 128-update mapper, whose measured 1.546 seconds of training counts toward each budget. Each worker stops after the first completed update reaching its remaining budget. Recorded overshoot is at most one update, ranging from 0.002 to 0.201 seconds across the twelve fits. The three four-GPU fitting jobs finish in 2m33s, 2m35s and 7m12s.

Table 25: Measured training costs and subsequent decoding in one common development campaign. Worker time includes model/cache setup, validation and export where performed, plus the measured initial mapper worker cost for CE and LoRA. It excludes Python startup and shared cache construction. Ratios compare against feature regression at the same warm budget.

Budget	Objective	Updates	Warm s	Worker s	Tok/s	Feature ratio [95% CI]
0.25×	Feature MSE	2026	22.86	52.04	145.05	1.000 [1.000, 1.000]
0.25×	Connector CE	106	22.94	91.89	111.04	0.766 [0.737, 0.791]
0.25×	LoRA32 (1729)	95	23.00	94.66	119.30	0.823 [0.786, 0.859]
0.25×	LoRA32 (1730)	94	22.86	95.20	119.52	0.824 [0.788, 0.860]
1×	Feature MSE	8144	91.42	119.71	144.80	1.000 [1.000, 1.000]
1×	Connector CE	474	91.58	161.04	109.60	0.757 [0.726, 0.786]
1×	LoRA32 (1729)	402	91.45	165.73	115.02	0.794 [0.757, 0.831]
1×	LoRA32 (1730)	407	91.50	164.61	115.49	0.798 [0.765, 0.830]
4×	Feature MSE	32446	365.65	390.11	144.55	1.000 [1.000, 1.000]
4×	Connector CE	1899	365.72	435.56	105.79	0.732 [0.699, 0.763]
4×	LoRA32 (1729)	1651	365.85	438.51	118.49	0.820 [0.786, 0.853]
4×	LoRA32 (1730)	1646	365.76	437.99	119.55	0.827 [0.793, 0.860]

Table 26: References measured within the same budget-decoding campaign. The initial mapper has 128 feature-regression updates. The 8,192-update reference is an existing saved checkpoint.

Reference	Tok/s	Initial mapper ratio [95% CI]
AR	28.61	0.240 [0.207, 0.277]
Native drafter	151.71	1.275 [1.205, 1.348]
Source reuse	84.71	0.712 [0.684, 0.737]
Initial mapper	119.01	1.000 [1.000, 1.000]
Dense, 8192 updates	145.82	1.225 [1.154, 1.302]

Feature regression reaches 145.05, 144.80 and 144.55 tokens/s at the three budgets. Relative to the quarter-budget fit, the 1-times and 4-times fits retain 99.83% [97.18, 102.53]% and 99.66% [97.35, 102.11]%, respectively. These intervals do not establish a benefit from longer fitting. Connector CE retains 73.2–76.6% of same-budget feature throughput, and the two LoRA seeds retain 79.4–82.7%. This result concerns the declared teacher-forced objective, initialization, rank and development-selected learning rates. It does not establish that other adaptation objectives or full drafter training fail.

Warm training is a marginal cost with cached features available. The historical full-cache extraction took 712.58 seconds and produced 292.8 GB. We reuse its first 512 training records and report the original construction cost separately. Dividing that cost by the record count would not measure a fresh 512-example extraction. The timed fits do not add training examples or repeat large-data scaling.

All seventeen methods decode the same sixteen exposed development questions with a 256-token cap. Tables 25 and 26 report descriptive paired request bootstrap intervals. They do not provide full-answer quality, untouched confirmation or corrections for selecting among multiple comparisons. Absolute throughput is interpreted within this common campaign and its deterministic training/runtime configuration.

D EXTERNAL BASELINES IN THEIR PUBLIC RUNTIMES

We evaluate SD^2's public frozen-drafter option and the released PARD parallel drafter on sixteen exposed MATH development requests with a 256-token cap. Each system has a correct cached AR control using its own target runtime and identical pretokenized inputs. These are short development comparisons. Below we separately evaluate capped answer quality. No untouched confirmation is included in these measurements.

SD^2 trains 402,665,472 steering parameters with merged guidance from target layers 3, 16 and 33 into all drafter layers. Inherited target and drafter weights remain frozen. Each of six objective/rate settings sees 512 distinct records once, using 128 batch-four updates and 95,835 non-padding supervised positions. Records are capped at 192 tokens. Training takes 47.0–48.6 seconds per fit. The initial TVD and KL settings also have exact fitting replicas, whose costs remain in the experiment record. This supervision differs from the feature-regression and last-15-position adaptation objectives in the preceding study.

Table 27: One-epoch SD^2 development comparison. Intervals resample paired requests 10,000 times and do not adjust for rate selection. Exact AR counts identical capped token sequences.

Setting	Tok/s	AR ratio [95% CI]	Independent ratio [95% CI]	Exact AR
Matched AR	12.64	1.000 [1.000, 1.000]	0.671 [0.622, 0.719]	16/16
Independent drafter	18.83	1.490 [1.391, 1.607]	1.000 [1.000, 1.000]	6/16
Zero guidance	18.56	1.469 [1.371, 1.584]	0.985 [0.983, 0.988]	6/16
TVD, 10^{-5}	18.38	1.454 [1.360, 1.567]	0.976 [0.955, 0.997]	4/16
KL, 10^{-5}	18.17	1.438 [1.341, 1.552]	0.965 [0.937, 0.993]	9/16
TVD, 4×10^{-6}	18.52	1.466 [1.370, 1.574]	0.984 [0.961, 1.006]	6/16
TVD, 2×10^{-5}	18.47	1.462 [1.365, 1.570]	0.981 [0.953, 1.011]	6/16
KL, 4×10^{-6}	18.66	1.477 [1.384, 1.588]	0.991 [0.962, 1.016]	5/16
KL, 2×10^{-5}	18.42	1.458 [1.363, 1.569]	0.978 [0.955, 1.003]	10/16

The public SD^2 evaluation configuration uses FP16 target storage, BF16 drafter and steering storage, and BF16 autocast under Transformers 4.52.4. We verify original FP32 training weights before conversion and separately fingerprint the BF16 inference tensors. The best fitted point, KL at 4×10^{-6}, retains 99.11% [96.19, 101.62]% of independent-drafter throughput. This screen does not establish a steering gain. We freeze this rate before checking additional epochs on the same small pool.

Table 28: Released PARD with zero new-target fitting, Transformers 4.51.3, BF16 weights, eager attention and static caches. The released checkpoint carries inherited proposer training.

Setting	Tok/s	AR ratio [95% CI]	Exact AR
Matched eager AR	31.27	1.000 [1.000, 1.000]	16/16
Released PARD	98.60	3.153 [2.831, 3.558]	4/16

Every SD^2 committed token matches its actual verifier decision, and immediate versus deferred observation preserves tokens and acceptance. Each timed PARD request has a separate verifier-observed replica with identical raw output and acceptance counts. PARD's measured generation omits that added observer. Both timers include prefills and cache setup, preserve raw EOS/cap overshoot and exclude detokenization. Earlier failed exact-AR gates remain failed. Controlled cache, query and attention replays expose changes in rounded target scores, while future-query perturbations preserve earlier logits in the tested calls. The sequence differences in these tables require separate task-quality evidence. Absolute speeds across these public runtimes do not isolate algorithm effects, and shared model residency does not measure isolated serving memory.

D.1 ADDITIONAL EPOCHS AND CAPPED ANSWER QUALITY

After the six-rate screen, we fix KL and the selected learning rate and fit for 1, 2, 4 and 8 epochs on the same 512 examples. These are four fresh fits, each with its own learning-rate decay horizon, rather than checkpoints from one training trajectory. The one-epoch fit exactly reproduces the selected earlier training fingerprint. All four endpoints are evaluated together on the same sixteen development requests, with the public SD^2 runtime and matched controls.

Table 29: SD^2 epoch budget at fixed 512 distinct examples. Training KL is the mean over updates in each final epoch. It is not held-out loss. Independent-drafter throughput is 18.75 tokens/s. Intervals resample paired requests and do not adjust for model selection.

Epochs	Fit seconds	Final-epoch training KL	Tok/s	Independent ratio [95% CI]
1	48.44	0.3923	18.56	0.9900 [0.9607, 1.0149]
2	97.01	0.2787	18.45	0.9841 [0.9639, 1.0033]
4	193.81	0.2027	18.39	0.9808 [0.9555, 1.0039]
8	388.08	0.1254	18.26	0.9738 [0.9488, 0.9997]

Training KL decreases from 0.3923 to 0.1254, while every fitted endpoint has a throughput point below the independent drafter. Additional epochs do not establish a decoding improvement in this fixed-pool setting. The highest-throughput fitted endpoint remains one epoch. We freeze this checkpoint before longer quality evaluation. This result concerns the tested frozen-drafter configuration, supervision and pool, and does not establish a general failure of SD^2 or its other training options.

Table 30: PARD capped answer quality on 128 exposed MATH development requests, after an eight-request pilot passed. Both methods use the same 2048-token cap and public BF16 eager runtime. Cap hits include all outputs reaching the cap. None ends in EOS exactly at the cap.

Setting	Tok/s	AR ratio [95% CI]	Correct	Cap hits
Matched eager AR	31.67	1.000 [1.000, 1.000]	103/128	11
Released PARD	105.63	3.335 [3.222, 3.451]	105/128	8

PARD has five beneficial and three adverse correctness discordances against AR, a difference of $+1.56$ percentage points. A conservative paired 95% interval is $[-6.34, 9.30]$ points: form exact 97.5% Clopper–Pearson intervals for each discordance probability and combine their opposite endpoints using a Bonferroni bound. This construction requires IID request pairs and a fixed candidate. It does not establish a one-percentage-point noninferiority margin. The descriptive paired bootstrap interval is $[-2.34, 5.47]$ points. Sparse or absent discordances can make such intervals misleadingly narrow. No untouched confirmation sample was generated in this study.

PARD matches AR's full capped token sequence on 28 of 128 requests. Its mean output length is 664.5 tokens versus AR's 690.4, so the 3.34-fold token-throughput ratio differs from the 3.47-fold request-time ratio. Every measured PARD request retains a separate, exactly matching verifier-observed replica. These are development outcomes under the specified runtime and scorer, not untouched confirmation or an isolated algorithm ranking across runtimes.

Table 31: Selected one-epoch SD^2 on the same 128 exposed MATH development requests, with a 2048-token cap. The public mixed-precision runtime and matched AR control are retained. Both have 11 cap-length outputs, none with EOS exactly at the cap.

Setting	Tok/s	AR ratio [95% CI]	Correct	Cap hits
Matched AR	12.48	1.000 [1.000, 1.000]	103/128	11
Selected SD^2	19.92	1.596 [1.560, 1.632]	103/128	11

SD^2 and matched AR both score 103 of 128, with three beneficial and three adverse correctness discordances. The conservative paired 95% interval is $[-7.04, 7.04]$ percentage points, which does not establish the one-point noninferiority margin. The descriptive paired bootstrap interval is $[-3.91, 3.91]$ points. Full capped sequences agree on 42 requests. Mean output lengths are 679.1 versus 680.9 tokens, so the 1.60-fold token-throughput and request-time ratios are close. Both shards retain the same selected training and inference fingerprints, and saved answers reproduce under the pinned scorer.

The public physical-buffer limit can end an untimed warmup before its token cap. We record this termination only during warmup. Measured requests must still reach EOS or the cap and pass all actual-verifier checks. A replacement eight-request pilot passed before the complete quality run. Failed preliminary attempts remain excluded from the reported timings. These results concern the tested frozen-drafter configuration and its public runtime, not all SD^2 training options.

E ACCEPTANCE AND NUMERICAL AGREEMENT

We pool all recorded proposal cycles rather than averaging each request's mean. This weights cycles equally. DFlash's recorded length is one already target-selected position plus the accepted draft prefix. EAGLE-3's length follows its shared decoder's progress count. These quantities measure work advanced per cycle, including target-supplied progress. A proposed-token acceptance fraction requires separating those positions and accounting for the number proposed.

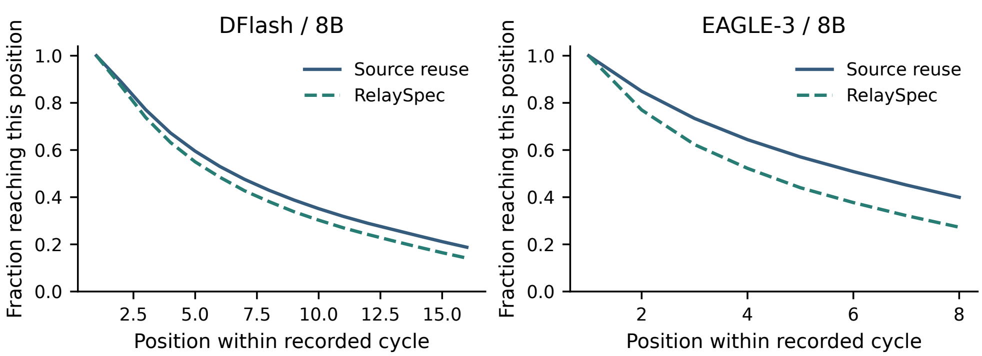

Figure 12: Historical 8B source-reuse comparisons. Each curve gives the fraction of recorded cycles reaching each position. Summing the curve values gives mean recorded progress per cycle. Solid and dashed lines show source reuse and RelaySpec, respectively.

The main MATH-500 table reports measured full-sequence agreement with AR. A fresh diagnostic selects eight prompts with early differences and saves target inputs, cache lengths, positions and the two leading logits at every call. The DFlash paths share the same first differing choice across native, source and relay drafting. The observed BF16 logits include ties or changed ordering with gaps up to 0.25 at those positions. Such traces identify numerical score differences in the selected cases. They do not establish that all mismatches have the same cause. The task-quality intervals in Table 4 remain the empirical quality evidence.

F DATA CHECKS AND DEVELOPMENT EXPOSURE

The 4,096-record fitting manifest contains distinct problems with solutions. The loader renders both fields before applying the 192-token cap. Repeating the manifest changes presentations, not distinct records. The 32-question development set informed normalization and objective choices. The historical 468-question complement is computed from saved full-run outputs and retains that exposure history.

Normalized exact matching finds zero overlaps between the 4,096 fitting and 1,250 evaluation problem texts. For a second check, each text is split into unique normalized tokens. Token-set Jaccard overlap is the number of shared tokens divided by the number of tokens in either set. Each evaluation question is compared with its closest fitting record. Table 32 reports the resulting similarities.

Table 32: Closest fitting-to-evaluation token-set overlap. Threshold columns count evaluation records at or above the stated overlap.

Task	Maximum overlap	≥ 0.80	≥ 0.90	≥ 0.95
MATH-500	0.980	47	13	4
GSM8K	0.378	0	0	0
HumanEval	0.290	0	0	0
MBPP	0.333	0	0	0
MT-Bench	0.571	0	0	0

MATH-500 includes 47 questions above 0.80 overlap and four above 0.95. These shared templates qualify the interpretation of math generalization. The fixed-map code and conversation evaluations give additional task coverage without changing the fitted map.

G Fitting time and isolated memory

Table 33: Selected map size and warm fitting-loop duration on four RTX 6000 Ada GPUs. Models are loaded before the recorded loop begins.

Drafter	Target	Trainable weights (millions)	Fitting loop (seconds)
DFlash	8B	52.43	85.6
DFlash	14B	65.54	118.0
EAGLE-3	8B	52.43	86.7
EAGLE-3	14B	65.54	118.5

The fitting-loop timer covers source and target feature computation and matrix optimization. A complete setup also loads model weights, writes the map, reloads it and initializes generation. Published original training counts in Table 34 provide context for the additional-data requirement. They are not a wall-time or token-budget comparison with fitting a new native drafter.

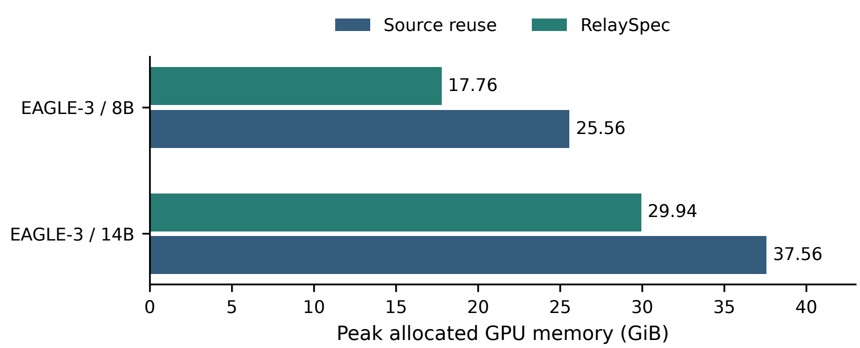

Figure 13: Isolated EAGLE-3 peak allocated memory on RTX 6000 Ada. Each method runs in a fresh process on eight paired prompts per target. Source removal saves 7.80 GiB at 8B and 7.63 GiB at 14B.

GPU peak counters are reset before the measured request. Allocated memory counts live tensor allocations and differs from reserved allocator memory. The mixed-arm processes used for timing are not the basis of this source-removal measurement.

H Additional related work

Improving drafting. EAGLE predicts future features with shifted tokens (Li et al., 2024a). EAGLE-3 combines feature layers and simulated drafting steps (Li et al., 2025). DFlash predicts a block in parallel (Chen et al., 2026), while Medusa uses several prediction heads (Cai et al., 2024). GRIFFIN addresses training-to-decoding token alignment (Hu et al., 2025). RepSpec changes the training structure (Huo et al., 2026), while VSD and Draft-OPD optimize accepted proposals (Zou et al., 2026; Lei et al., 2026). RelaySpec inherits the trained drafter and changes only its conditioning input.

Proposal structure and execution. EAGLE-2 and OPT-Tree adapt proposal trees (Li et al., 2024b; Wang et al., 2025). SpecInfer organizes tree verification (Miao et al., 2024). Distributed speculative inference and speculative speculative decoding overlap computation (Timor et al., 2025b; Kumar et al., 2026). Draft & Verify skips target layers (Zhang et al., 2024), LayerSkip trains early exits (Elhoushi et al., 2024), and Lookahead checks candidates without an auxiliary drafter (Fu et al., 2024). RelaySpec retains an external drafter and holds its released proposal structure fixed. We evaluate the cost and accepted progress of changing its feature source.

I Inherited drafter training context

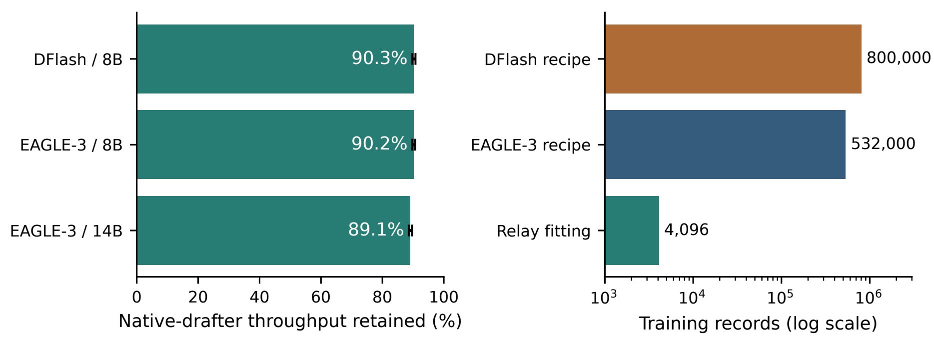

Figure 14: Left: fraction of native target-specific drafter throughput retained by RelaySpec in paired MATH-500 runs. Right: published original DFlash and EAGLE-3 recipe sizes versus the additional RelaySpec fitting set. Recipe counts provide training-scale context, not a matched training experiment or an audit of each released checkpoint.

Table 34: Original drafter training and additional target adaptation. Record counts describe different training stages and objectives.

Stage	Records	Relative to relay fit	Weights learned
Published DFlash recipe (Chen et al., 2026)	∼800,000	∼195×	Drafter
Published EAGLE-3 recipe (Li et al., 2025)	∼532,000	∼130×	Drafter
RelaySpec target adaptation	4,096	1×	Feature map

The DFlash recipe reports about 800K target-generated records. The EAGLE-3 recipe uses about 68K ShareGPT and 464K UltraChat entries. Relay fitting uses 0.51% and 0.77% of these respective record counts. This comparison describes marginal adaptation: the original drafter's training remains inherited. Record lengths, objectives and hardware differ, so these ratios do not represent equivalent reductions in training compute. The evaluated EAGLE-3 checkpoints come from DeepSpec, while the table's EAGLE-3 count refers specifically to the published training recipe.